

AI Agents and Search

Lara J. Martin (she/they)

<https://laramartin.net/neurosymbolic-text-gen>

Slides adapted from Chris Callison-Burch, Cynthia Matuszek, & Cassandra Kent

Learning Objectives

Understand the difference between traditional AI agents and agentic AI

Categorize agents based on their capabilities and behavior

Formulate a search problem

Review basic types of tree search algorithms

A word cloud featuring various terms related to social media and technology. The words are arranged in a dense, overlapping manner, with some words appearing larger and bolder than others. The terms include: WEB, SOCIAL, DATA, NEWS, MOBILE, FLICKR, SEARCH, PEOPLE, LIKE, SHARE, CONTENT, RADIO, NETWORK, ONLINE, MONEY, INTERNET, COMMUNITY, INFORMATION, POPULAR, FRIENDS, MEDIA, GLOBAL, YOUTUBE, FACEBOOK, GOOGLE+, WORDPRESS, COMMUNICATION, and MOBILE. The words are in different shades of gray and black, creating a textured, layered effect.

```
graph TD; 4[4 Input] --> 3[3]; 2[2 Query] --> Index[Index]; Index --> 5[5]; 3 --> Plus[+]; 5 --> 6[6 Retrieved Text Snippet];
```

4 Input

2 Query

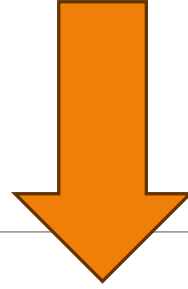
3

Index

5

Retrieved Text (Snippet)

Where are we?



Module 1

- Neural Networks
- Language Modeling

Module 2

- Introducing LM-based guided generation
- Scripts, Events, Procedures, Plots
- RAG

Module 3

- Search & Planning
- RL

Module 4

- Commonsense reasoning tasks
- Knowledge Bases, Ontologies
- World Tracking

Module 5

- Human-in-the-loop systems
- Dialog
- Simulation
- Personas

AI Agents

AI Agent Definition

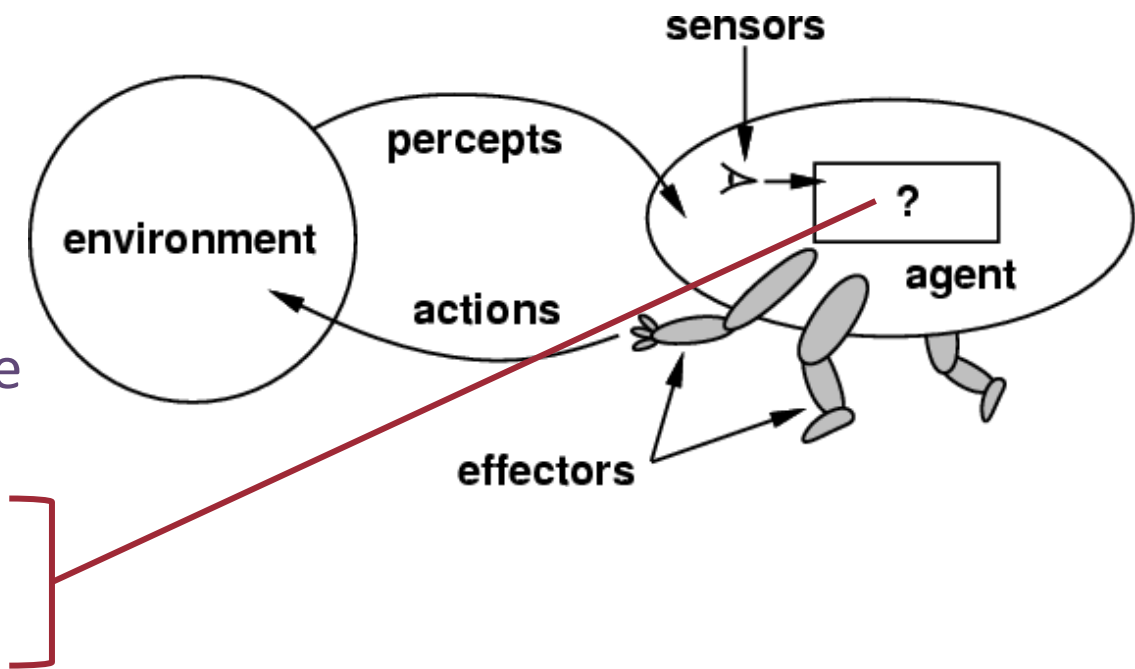
Agent: anything that **perceives** its environment through **sensors**, and **acts** on its environment through **actuators**

Percept: input at an instant

Percept sequence: history of inputs

Agent function: mapping of **percept sequence** to **action**

Agent program: (concise) implementation of an agent function



Picture from Dr. Cynthia Matuszek, source unknown

“Agentic AI”

Term coined around 2023 but has become popular as of early 2025

Using a *language model* to behave like an “agent” – the intersection of AI & NLP beyond chatbots

“autonomous systems designed to pursue complex goals with *minimal human intervention*” [1]

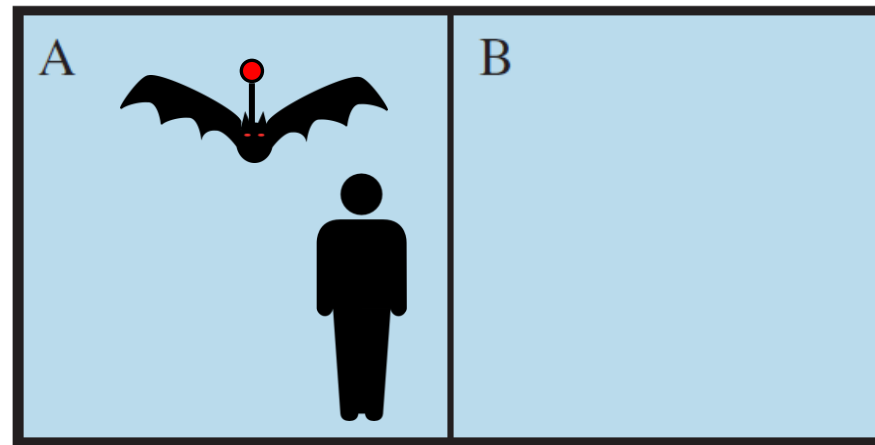
But technically all AI is agentic...

Types of Agents

Similar to environments, we can come up with a taxonomy of agents based on how they behave:

1. Simple Reflex Agent
2. Model-based Reflex Agent
3. Model-based Goal-based Agent
4. Model-based Utility-based Agent

Example: Vampire World



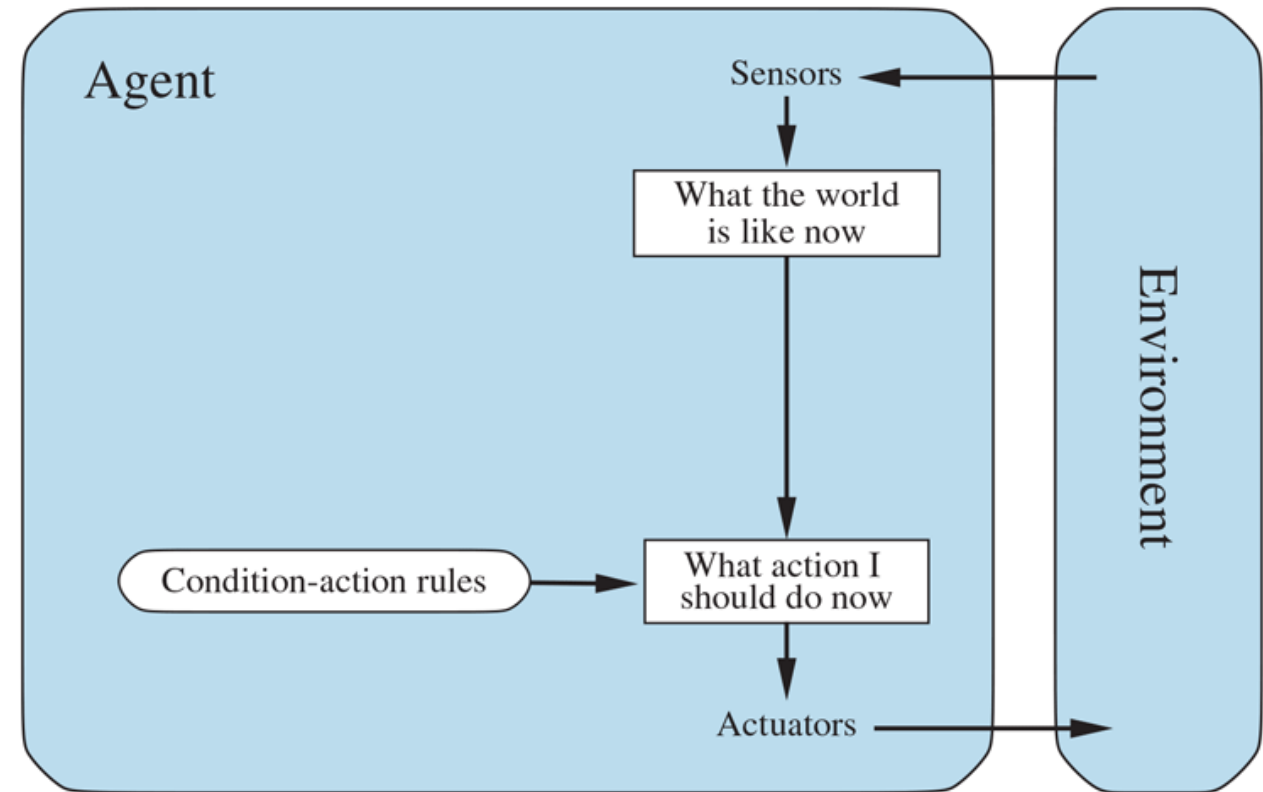
Types of Agents:

Simple Reflex Agent

Decide actions directly from the current percept sequence

Example:

```
if [A, Empty]: return Right
if [B, Empty]: return Left
if [A or B, Human]: return Suck
```



Artificial Intelligence: A Modern Approach, Pg. 50

Types of Agents:

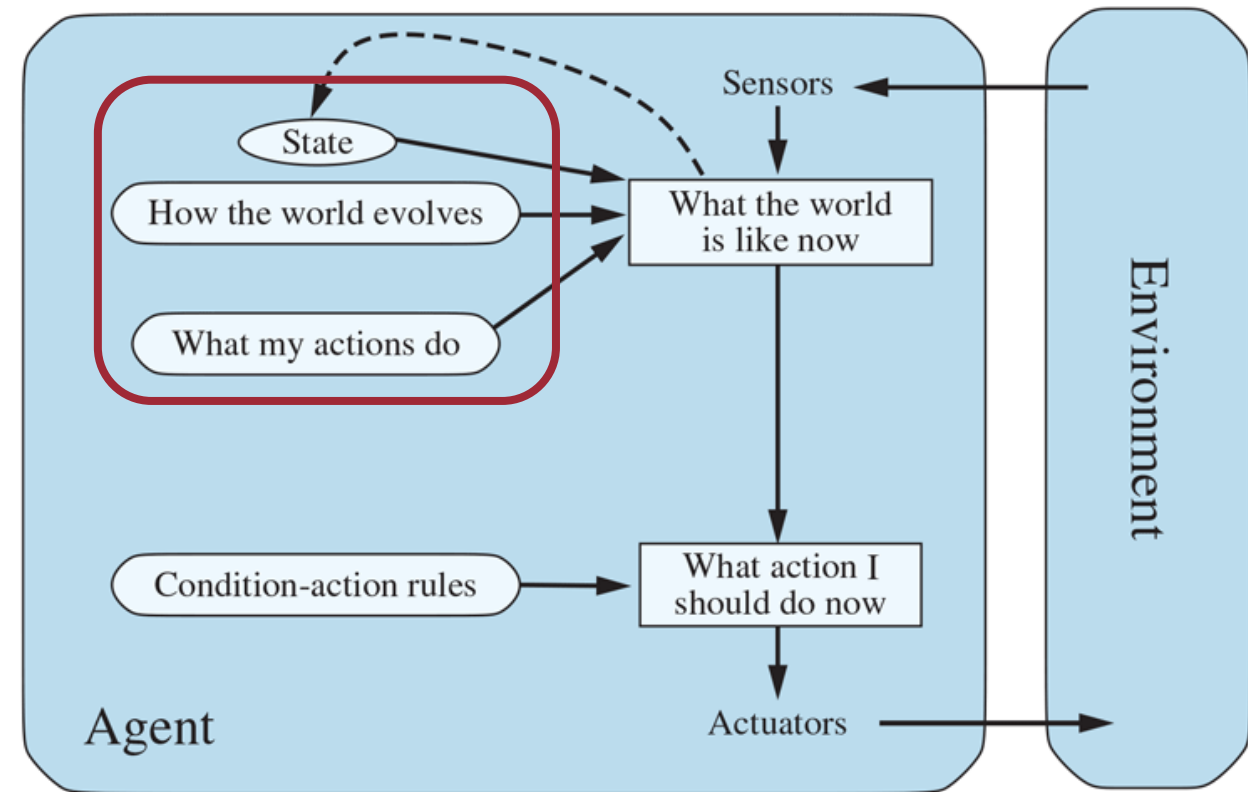
Model-based Reflex Agent

Attempt to model the state of the world, decide action based on modeled (“remembered”) state

Example:

Observe that at a certain time, people are more likely to show up in A

Move to location based on time of day



Artificial Intelligence: A Modern Approach, Pg. 52

Types of Agents:

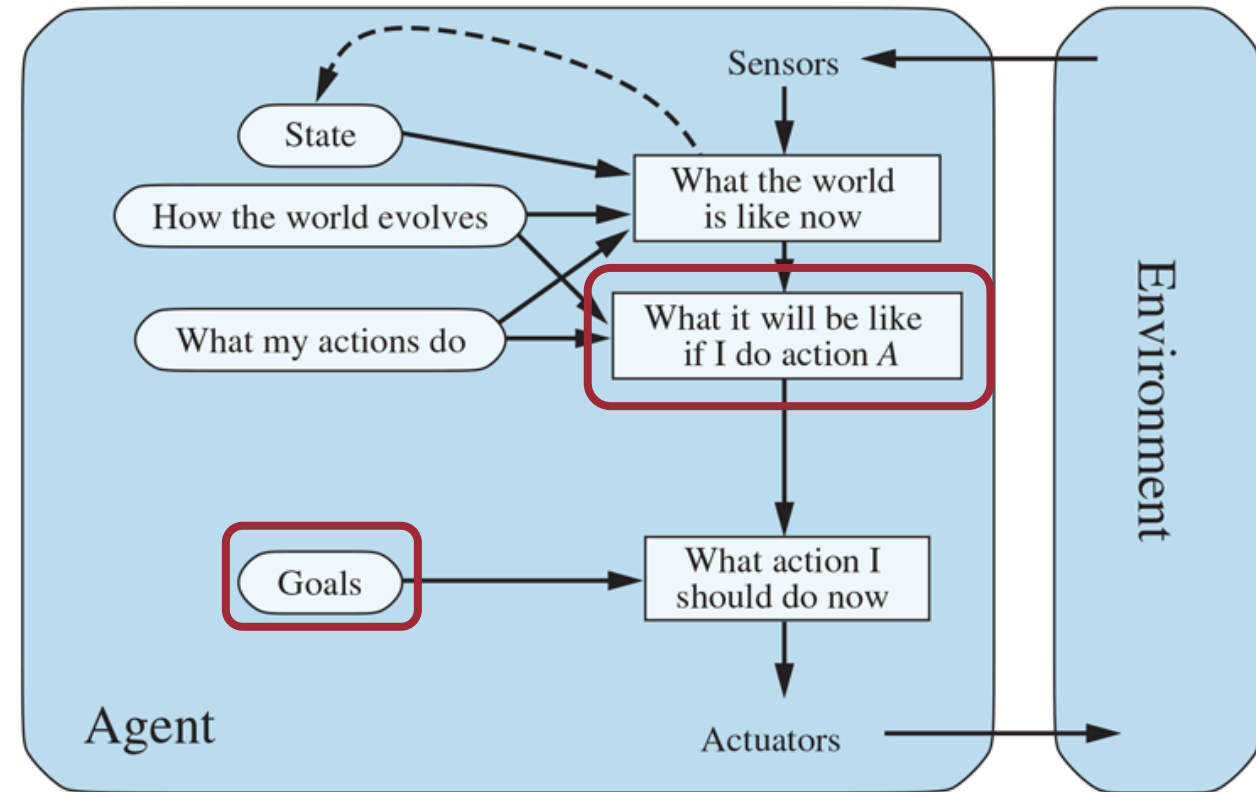
Model-based Goal-based Agent

Action choices change as the goal changes

Example:

For the same performance criteria (suck as much blood as possible), we can add a goal of keeping one room clear to act as a home

As the goal changes, vampire changes how often it moves



Artificial Intelligence: A Modern Approach, Pg. 53

Types of Agents:

Model-based Utility-based Agent

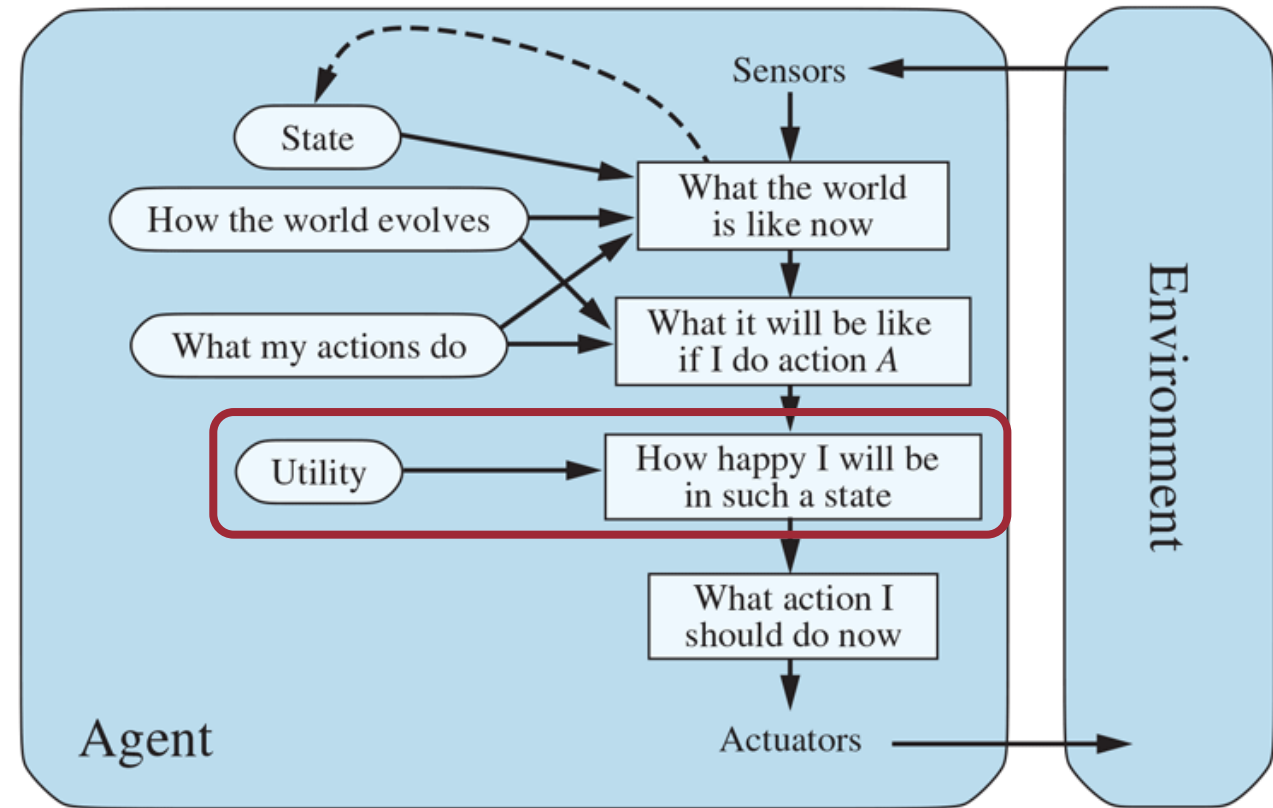
Agent cares not only about reaching a goal, but also *how* it reaches its goal

Example:

Energy-efficient vampire agent

Still wants to suck as much blood as possible

Limits how often it moves



Artificial Intelligence: A Modern Approach, Pg. 55

What type of agent is a Roomba?



1. Simple Reflex Agent

✓ 2. Model-based Reflex Agent ← Keeps a memory of the world but only reacts to its surroundings

✓ 3. Model-based Goal-based Agent ← Keeps a memory of the world and tries to visit all of the room

✓ 4. Model-based Utility-based Agent ← Optimizing battery life and when to go back to hub

What type of agent is Watson?



<https://cdn.geekwire.com/wp-content/uploads/2017/04/IBM-Watson.jpg>

Watson was an agent made by IBM that played *Jeopardy!* (and did quite well) in 2011

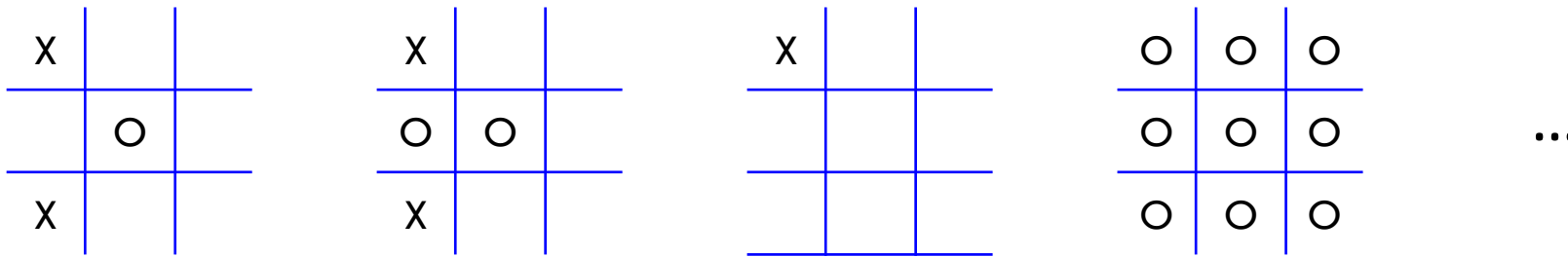
1. Simple Reflex Agent
2. Model-based Reflex Agent
3. Model-based Goal-based Agent
- ✓ 4. Model-based Utility-based Agent

← Not just to win but to get the most money

Search

Why Search?

Some problems are small enough to go through all possible states:



Tic-tac-toe: $3^9 = 19,683$ states (3 values for each cell, nine cells)

(With fewer *valid* states)

Most real-world problems are intractable

- We need to be smart about how we visit states

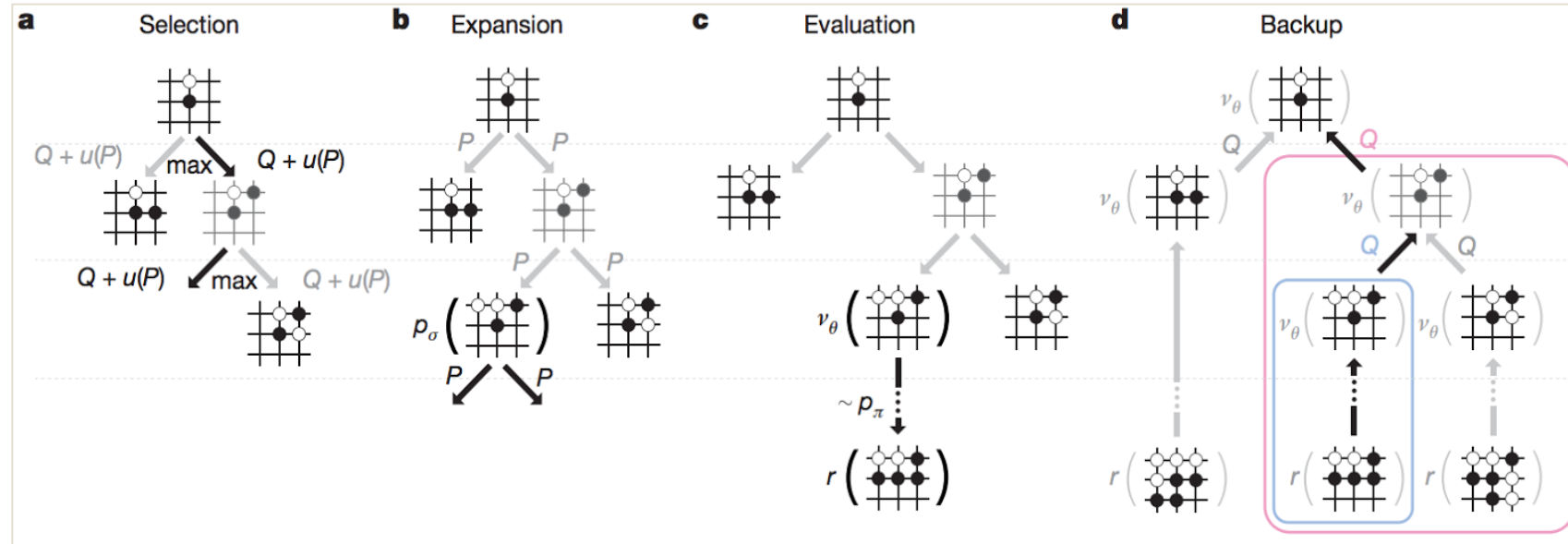


examples.gurobi.com/traveling-salesman-problem

Why Search?

For when we don't know how to reach a goal (or **any** goal) from a given initial state (or **any** initial state)

Search Today: AlphaGO



Pictures from needpix.com, machinelearnings.co/understanding-alphago-948607845bb1

Search Today: Story Generation

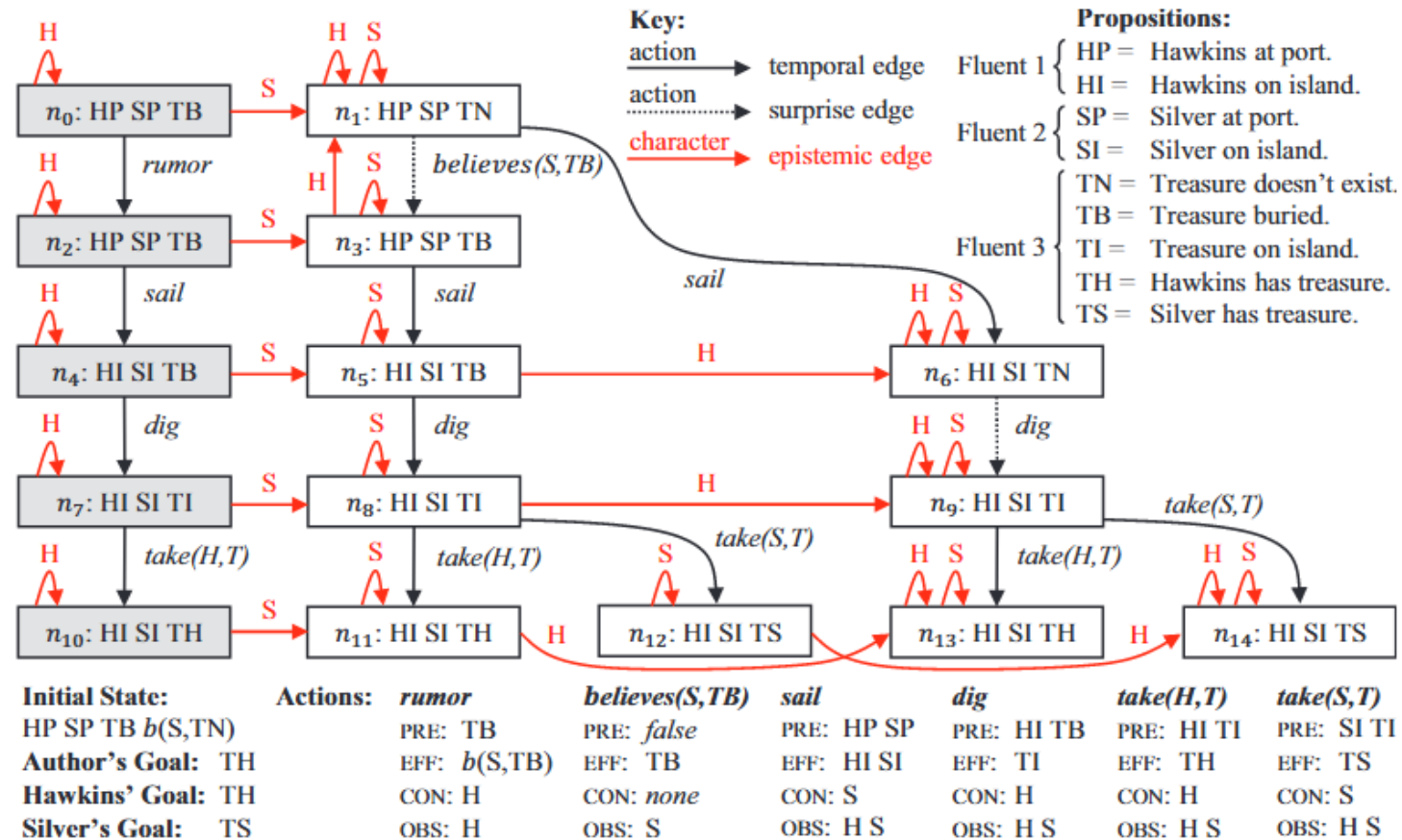


Figure 1: A narrative search space graph for the plot of *Treasure Island* (Stevenson 1919).

Defining a Search Problem

Break down the problem space into **states** and actions (for each state)

+ these components:

- Initial state – where the agent starts
- Transition model – how actions change states
- Goal test – measuring if the current state is the goal
- Step cost (optional) – how expensive it is to take action a in state s

Problem: Find a sequence of **actions** that **transitions** the **initial state** into a state which passes the **goal test**

Solution: sequence of actions, or a **plan**

Formal Definition of a Search Problem

1. **States:** a set S

2. **Initial state:** $s_i \in S$

3. **Actions:** a set A

$\forall s$ **Actions(s)** = the set of actions that can be executed in s .

4. **Transition Model:** $\forall s \forall a \in \text{Actions}(s)$

Result(s, a) $\rightarrow s_r$

s_r is called a **successor** of s

$\{s_i\} \cup \text{Successors}(s_i)^* = \text{state space}$

5. **Path cost** (Performance Measure):

Must be additive, e.g. sum of distances, number of actions executed, ...

c(x,a,y) is the step cost, assumed ≥ 0

◦ (where action **a** goes from state **x** to state **y**)

6. **Goal test:** **Goal(s)**

s is a goal state if **Goal(s)** is true.

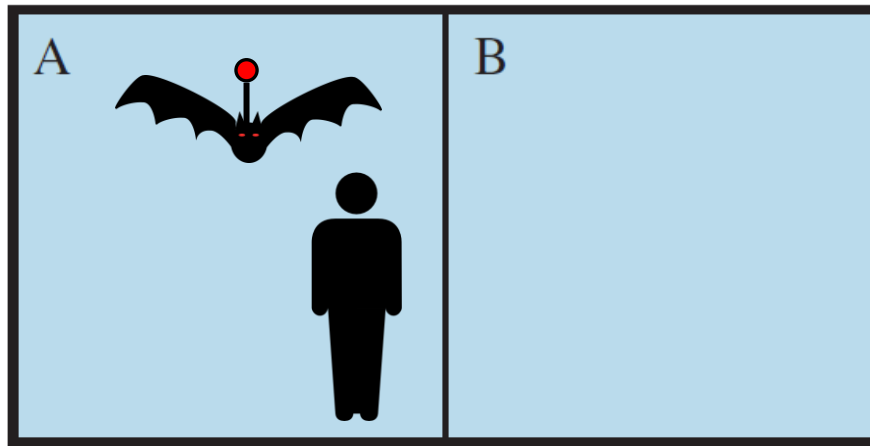
Can be implicit, e.g. **checkmate(s)**

What are States?

Current position and attributes of everything in the environment

- Only need the things relevant to the agent's decision making

Like discrete snapshots over time



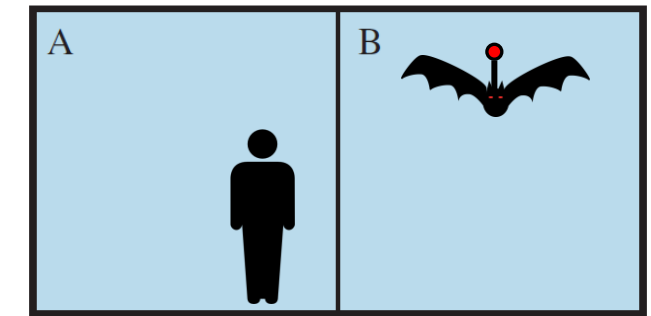
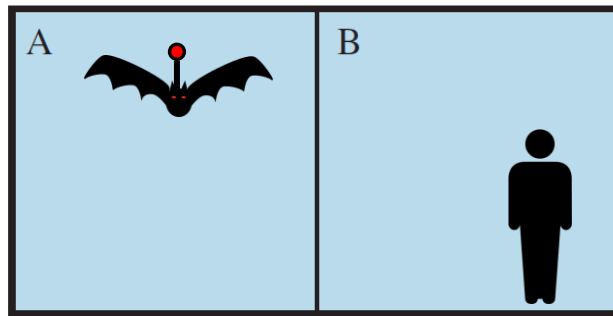
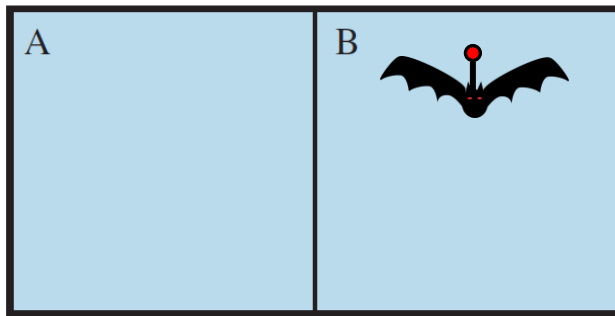
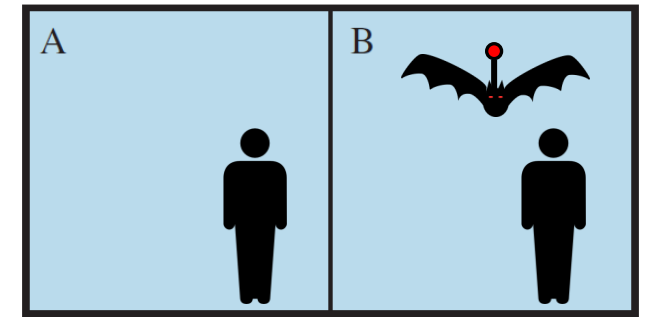
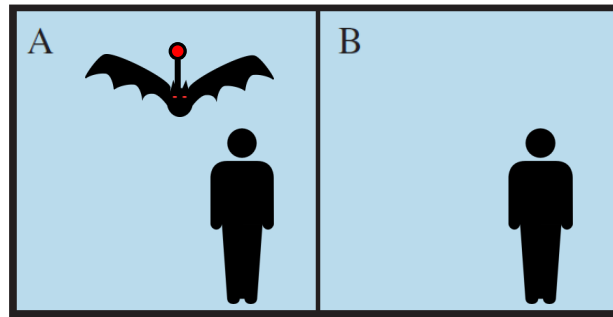
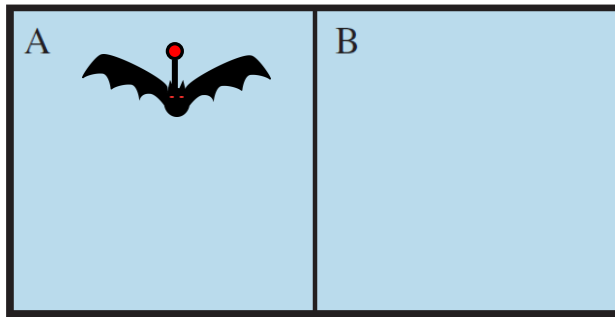
State

Location A: vampire, human

Location B: empty

Vampire World: State Examples

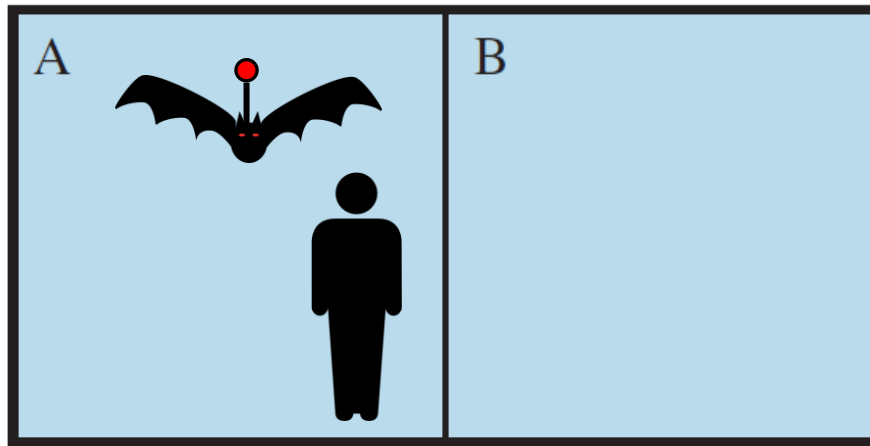
States: A snapshot of the world says which entities are in which cells.



And so on...

What Are Actions?

Anything the agent can do to affect the environment



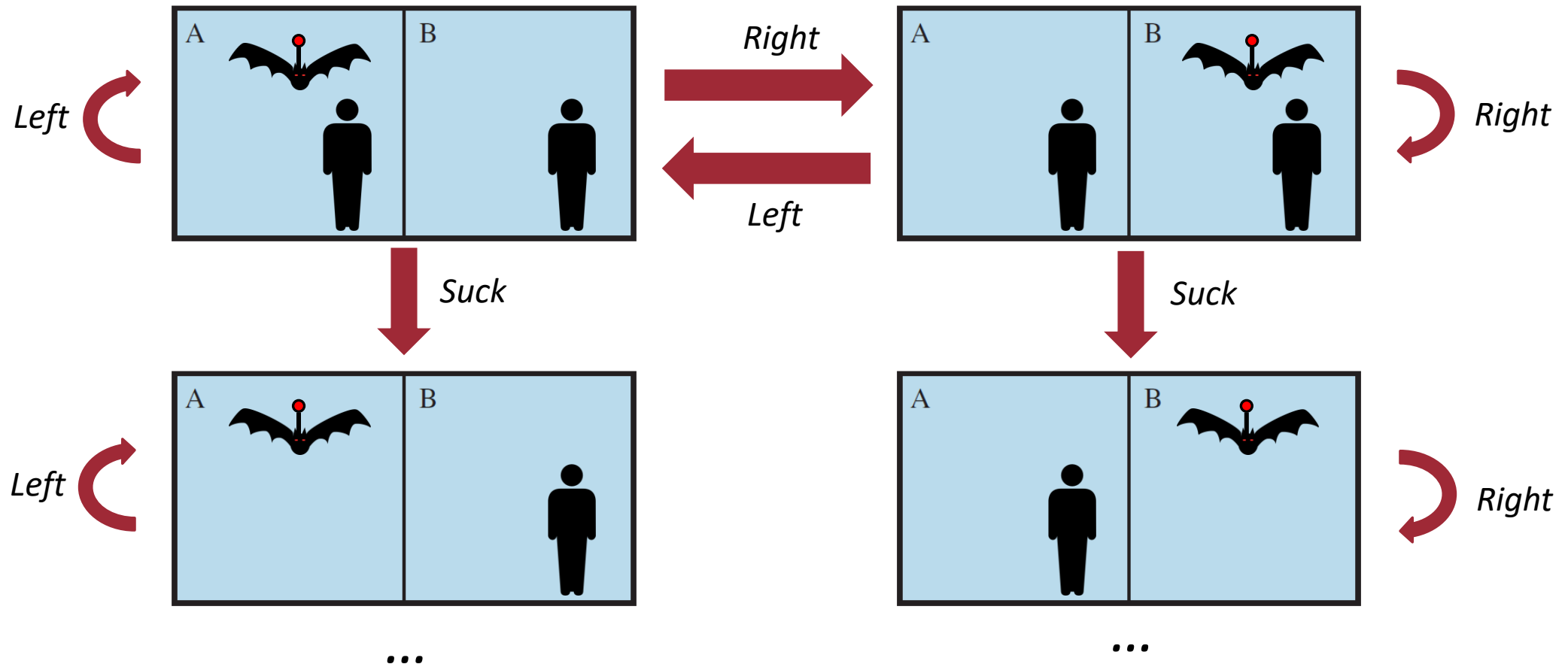
Actions

move left, move right, suck

Or for this particular state...

move right, suck

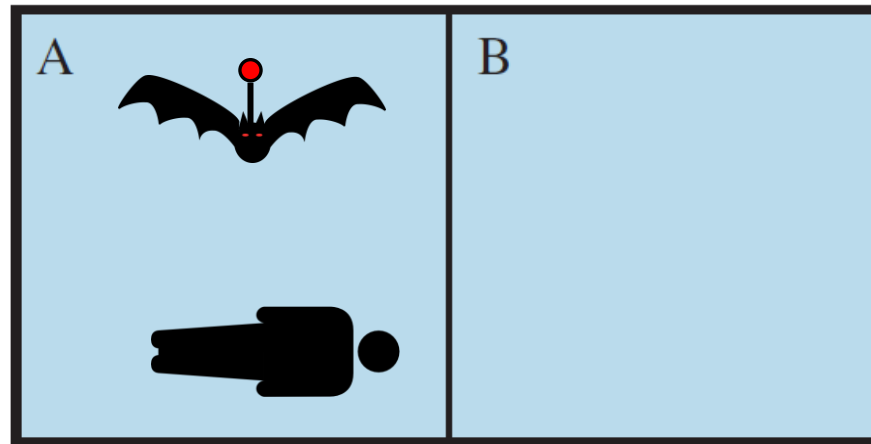
Vampire World: Transition Model Example



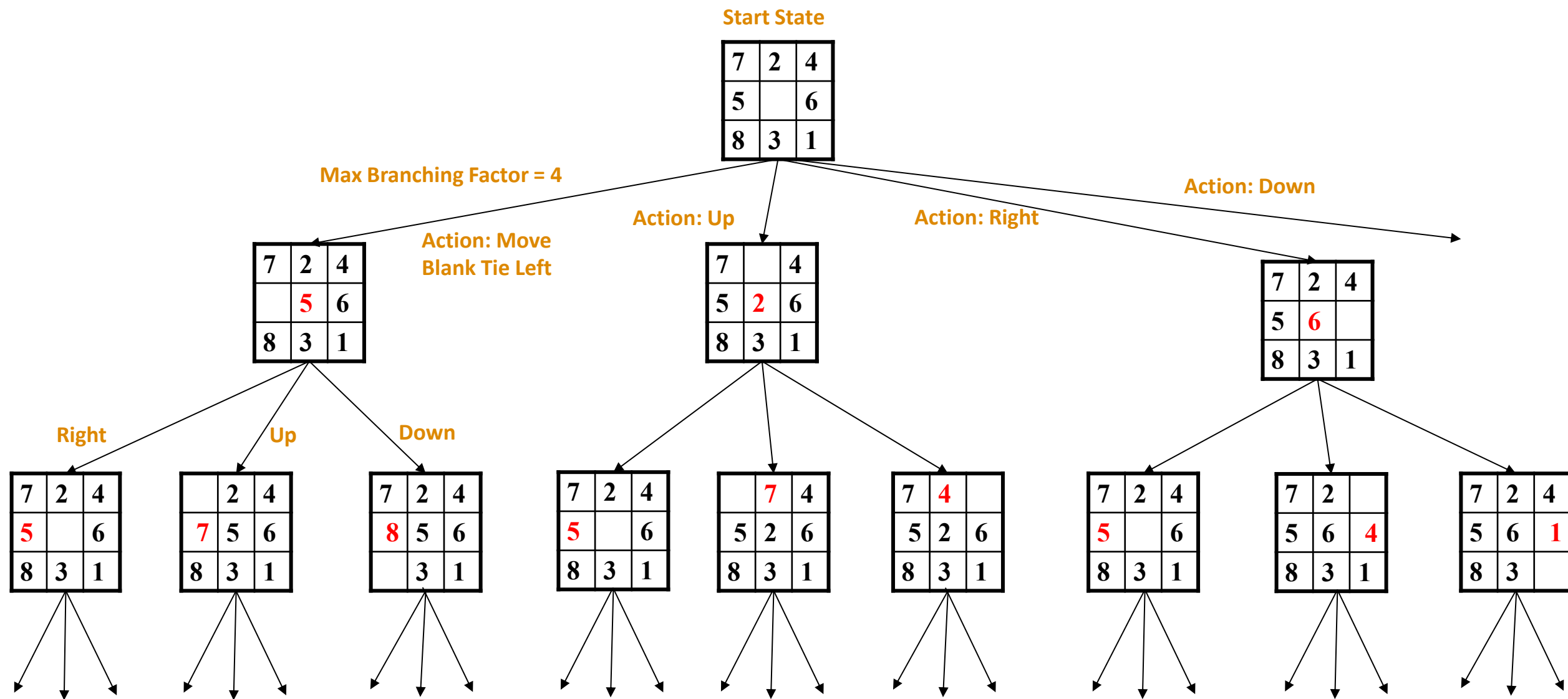
Goal Test

Measuring to see if the goal has been achieved.

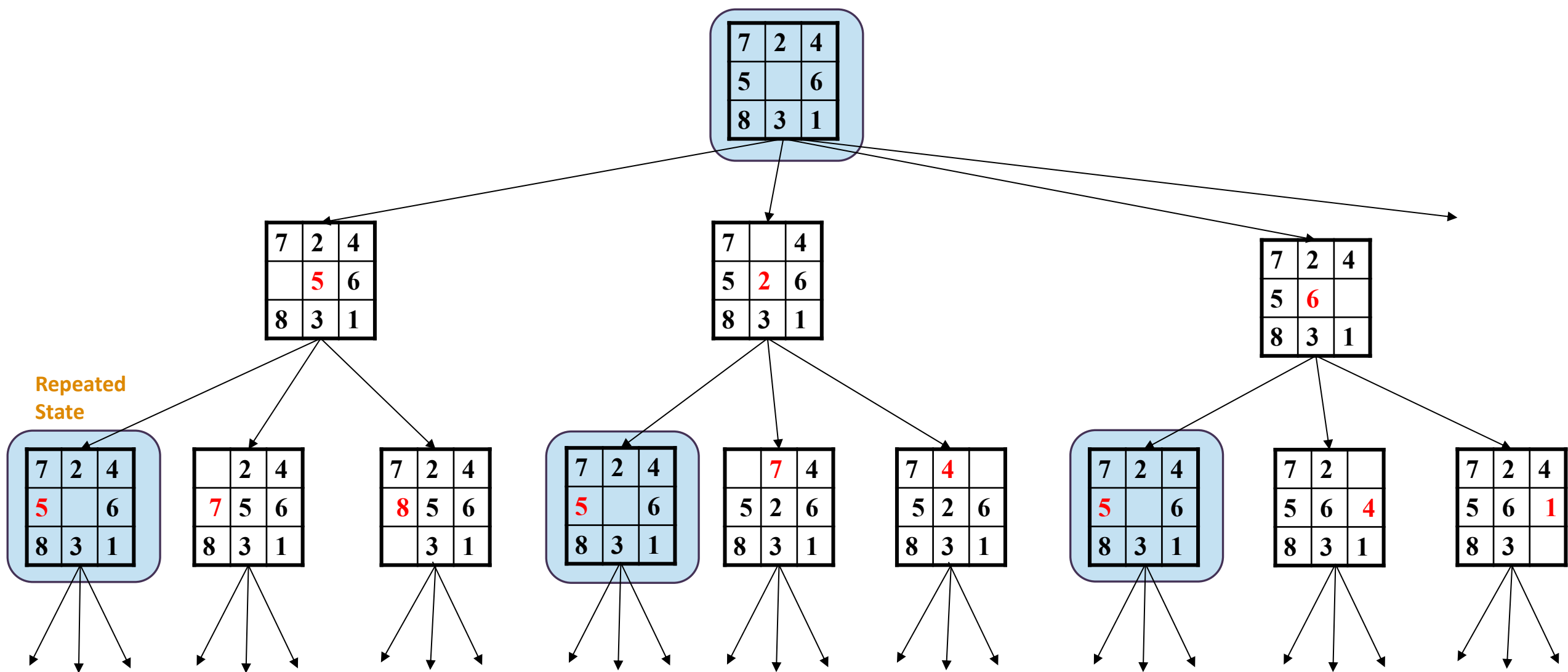
If our goal is for the vampire to suck a human's blood, then we need a way of measuring to see if that occurred.



Sliding Number Puzzle *Search Tree*



Sliding Number Puzzle *Search Tree*



Useful Concepts

State space: the set of all states reachable from the initial state by *any* sequence of actions

- When several operators can apply to each state, this gets large very quickly
- Might be a proper subset of the set of configurations

Path: a sequence of actions leading from one state s_j to another state s_k

Solution: a path from the initial state s_i to a state s_f that satisfies the goal test

Search tree: a way of representing the paths that a search algorithm has explored. The root is the initial state, leaves of the tree are successor states.

Frontier: those states that are available for *expanding* (for applying legal actions to)

Solutions and *Optimal* Solutions

A **solution** is a sequence of **actions** from the **initial state** to a **goal state**.

Optimal Solution: A solution is **optimal** if no solution has a lower **path cost**.

Basic search algorithms: Tree Search

Generalized algorithm to solve search problems

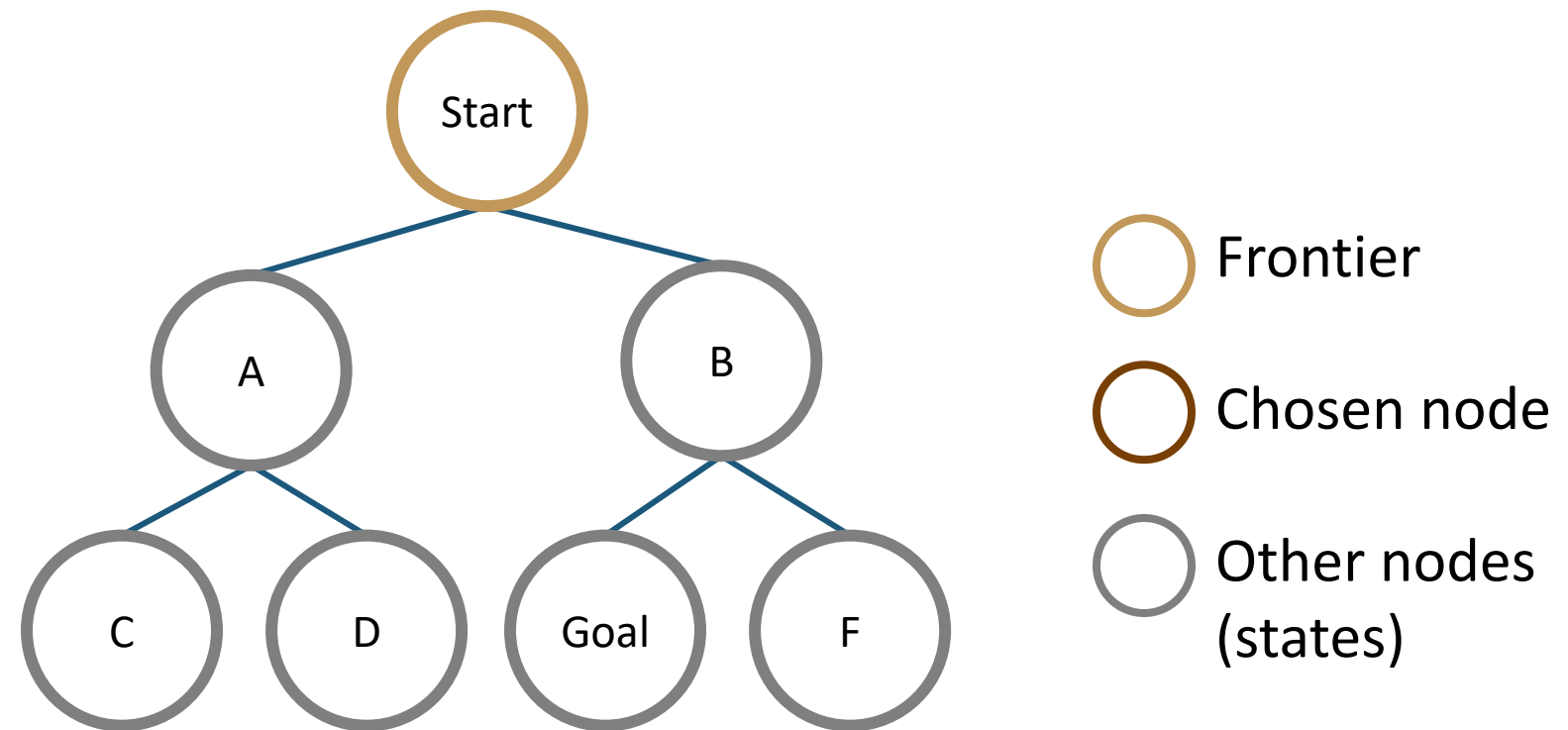
Enumerate in some order all possible paths from the initial state

- Here: search through *explicit tree generation*
 - ROOT= initial state
 - Nodes in search tree generated through **transition model**
 - Tree search treats different paths to the same node as distinct

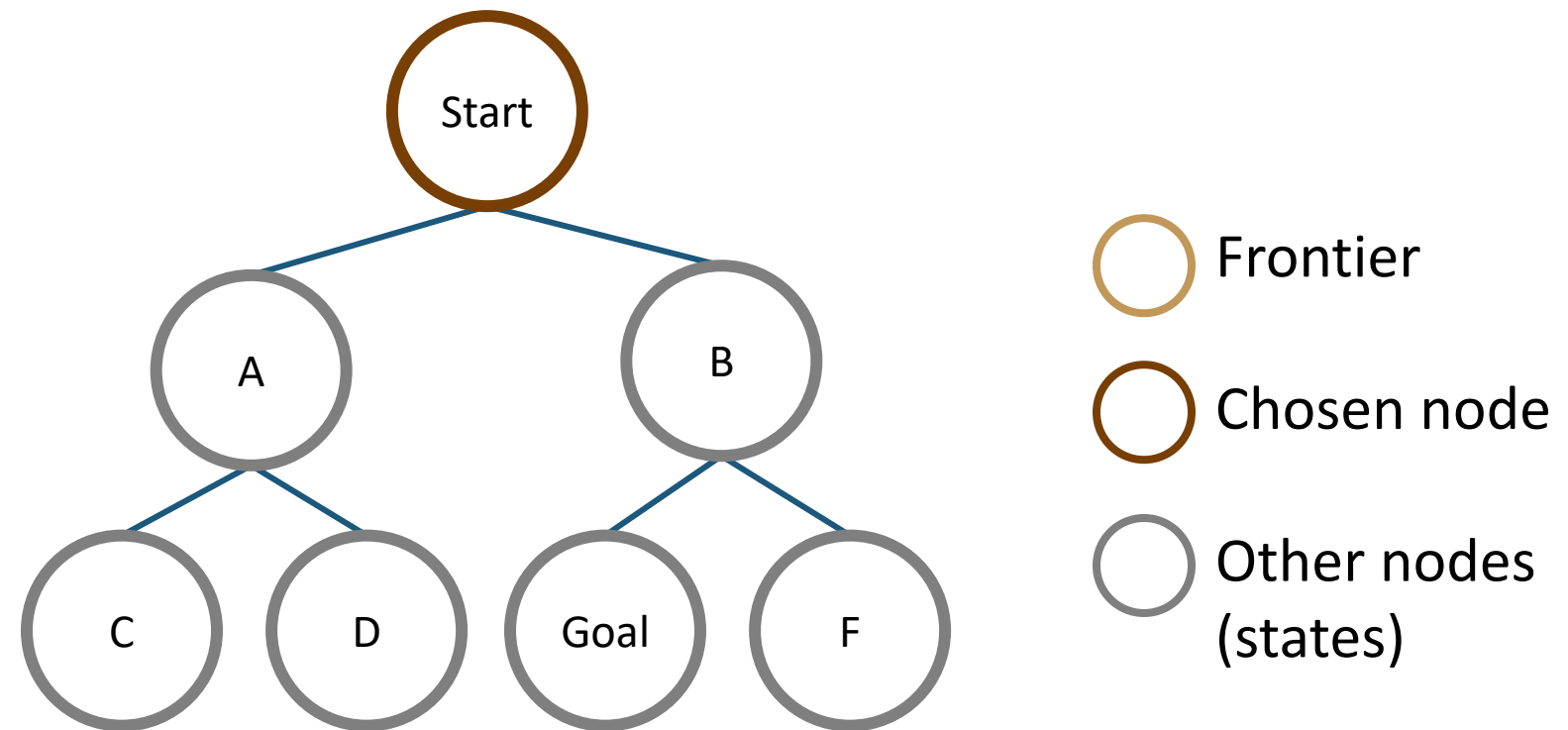
Generic Search Algorithm

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

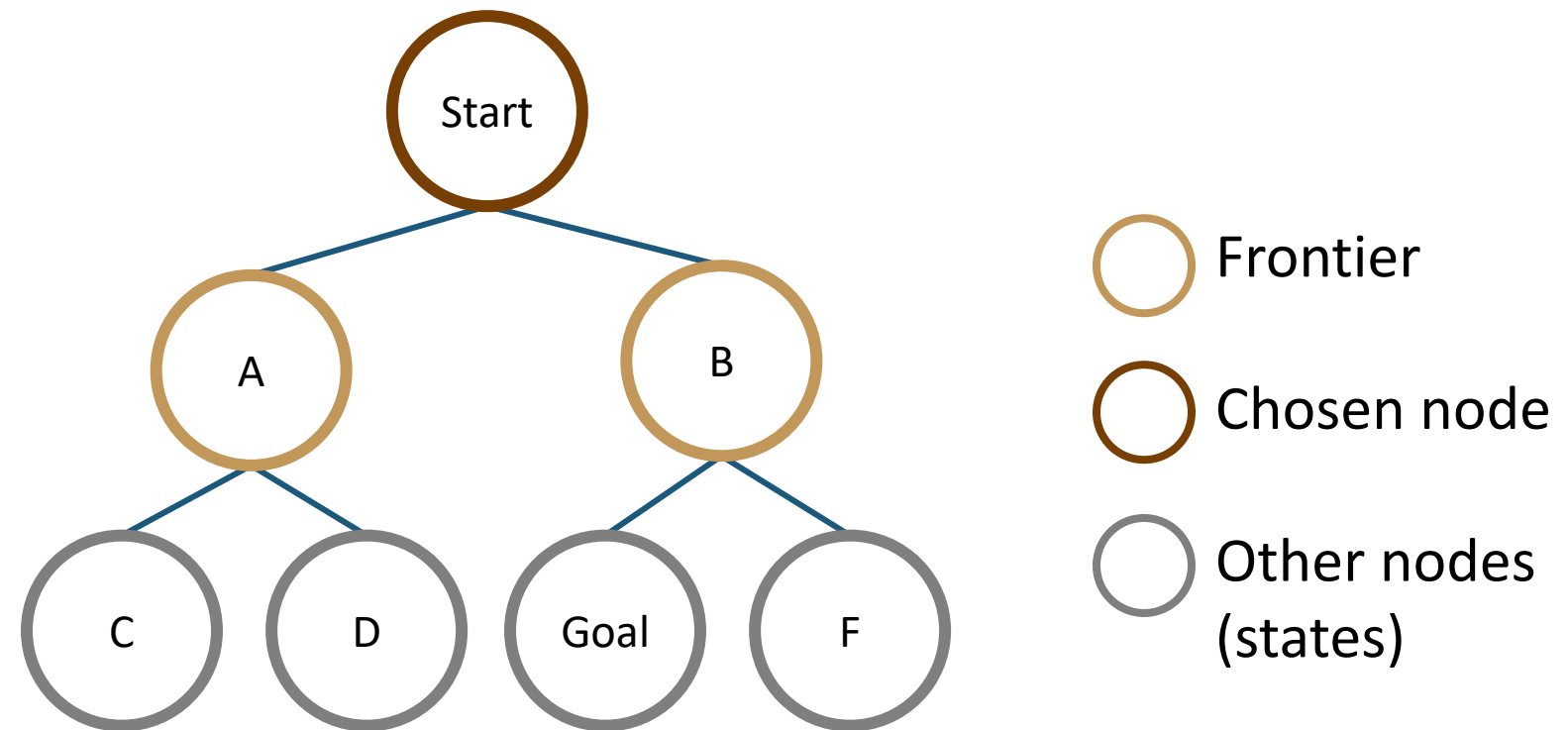
The Generic Search Algorithm



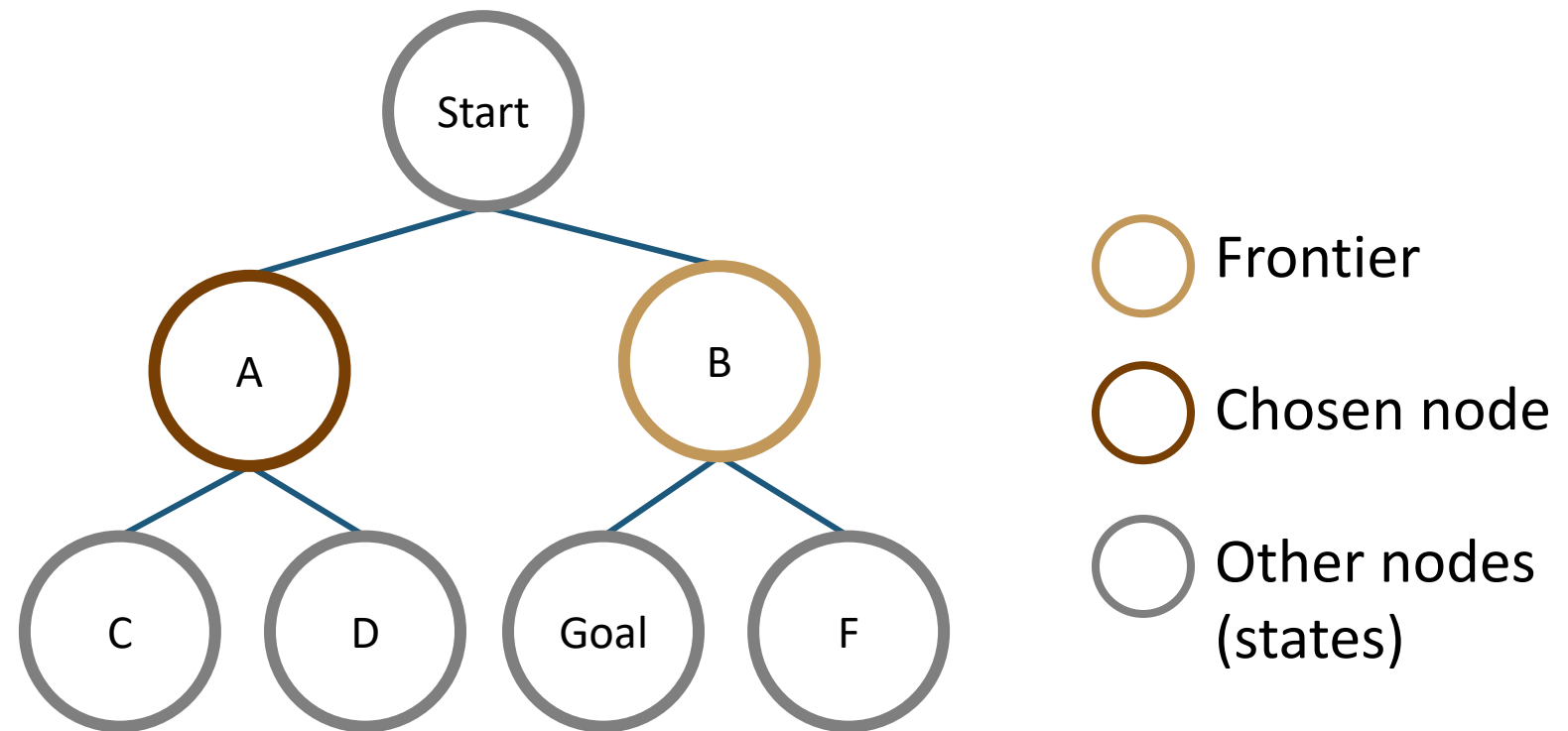
The Generic Search Algorithm



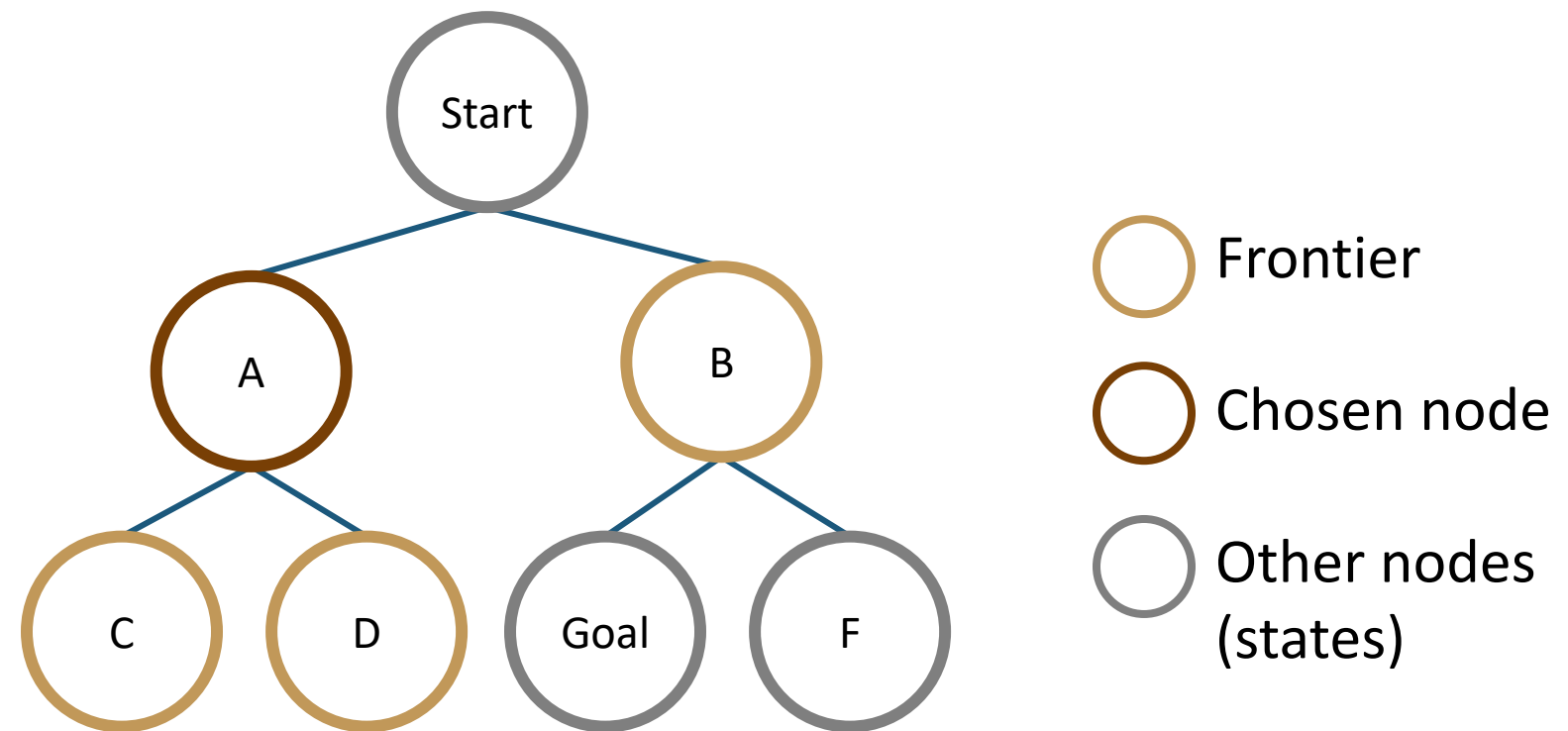
The Generic Search Algorithm



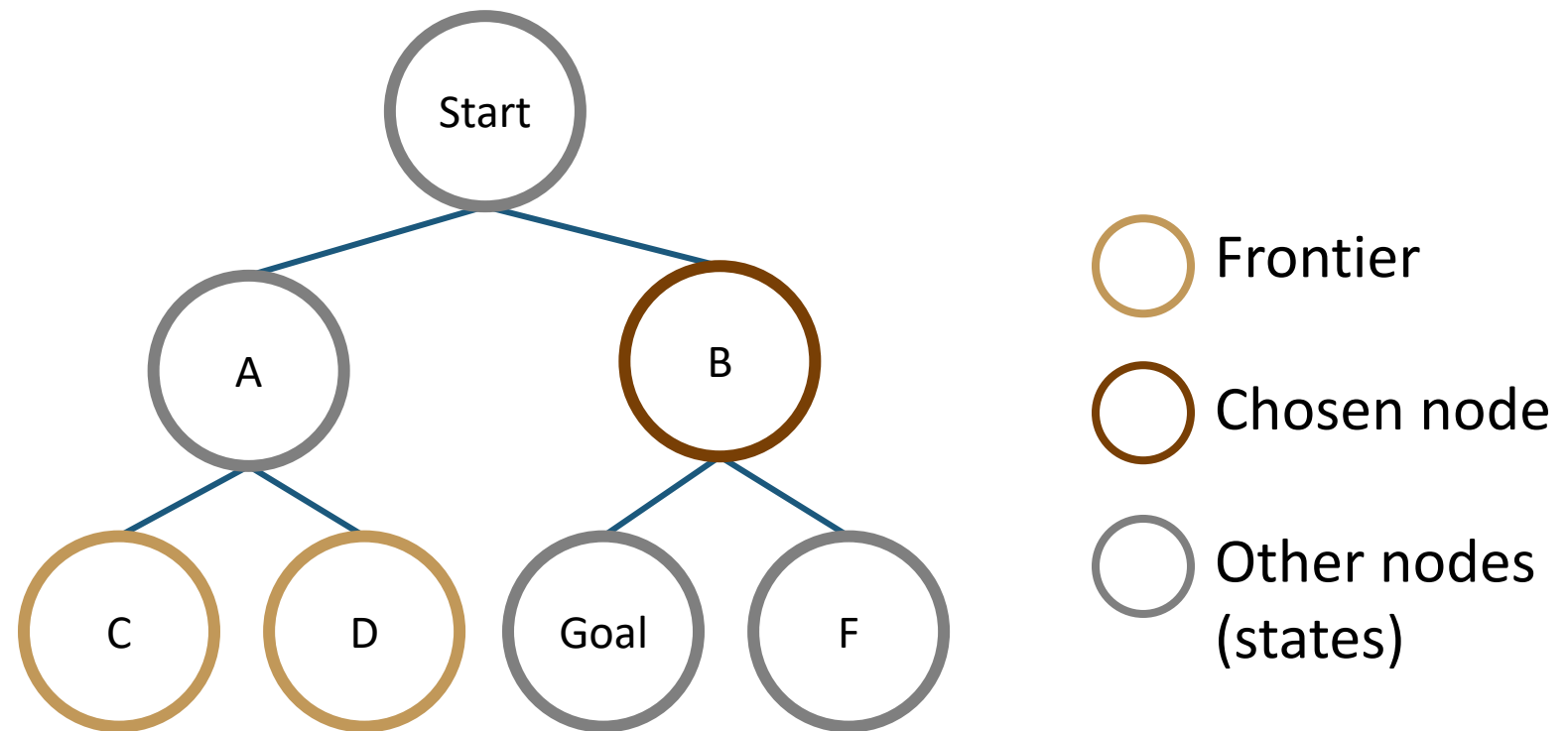
The Generic Search Algorithm



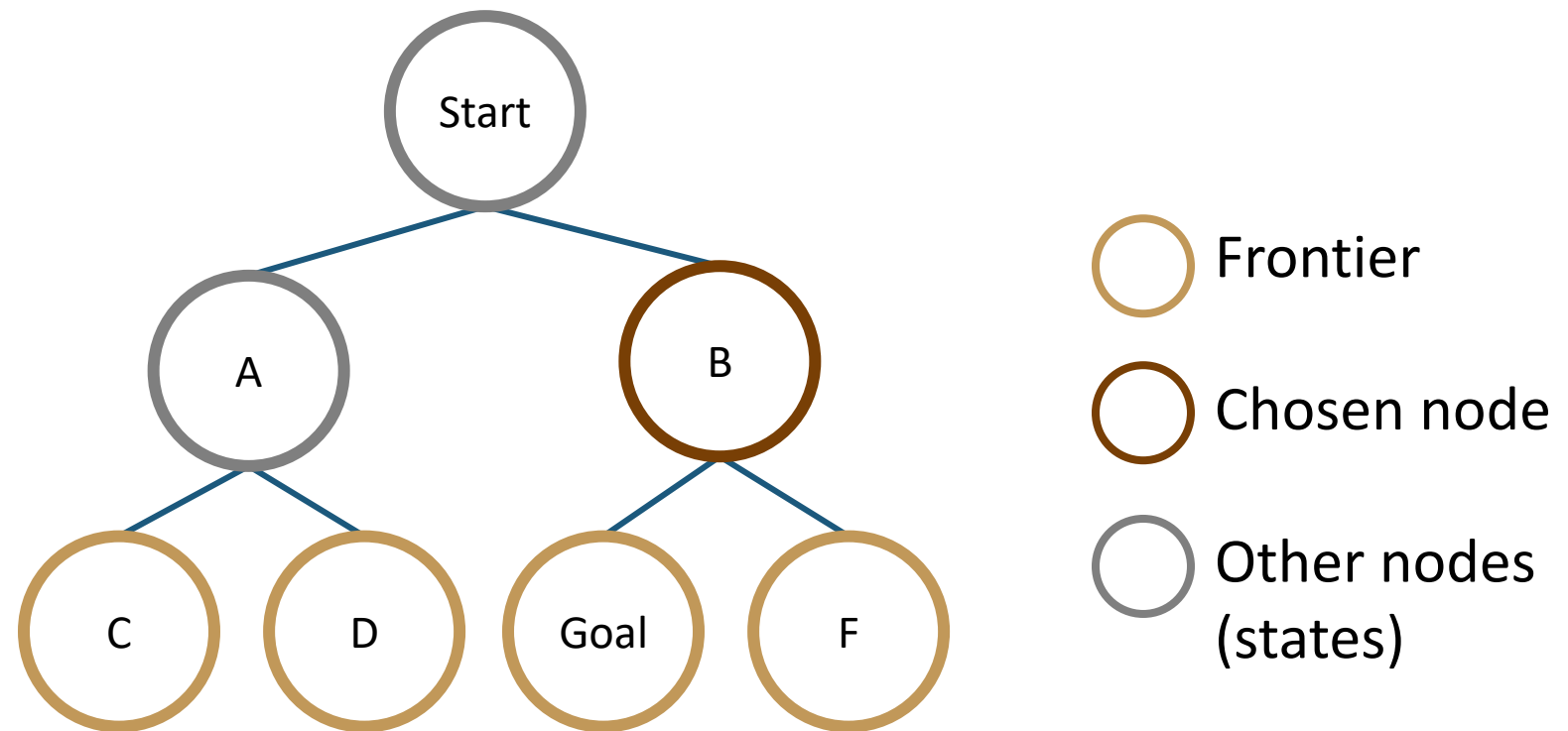
The Generic Search Algorithm



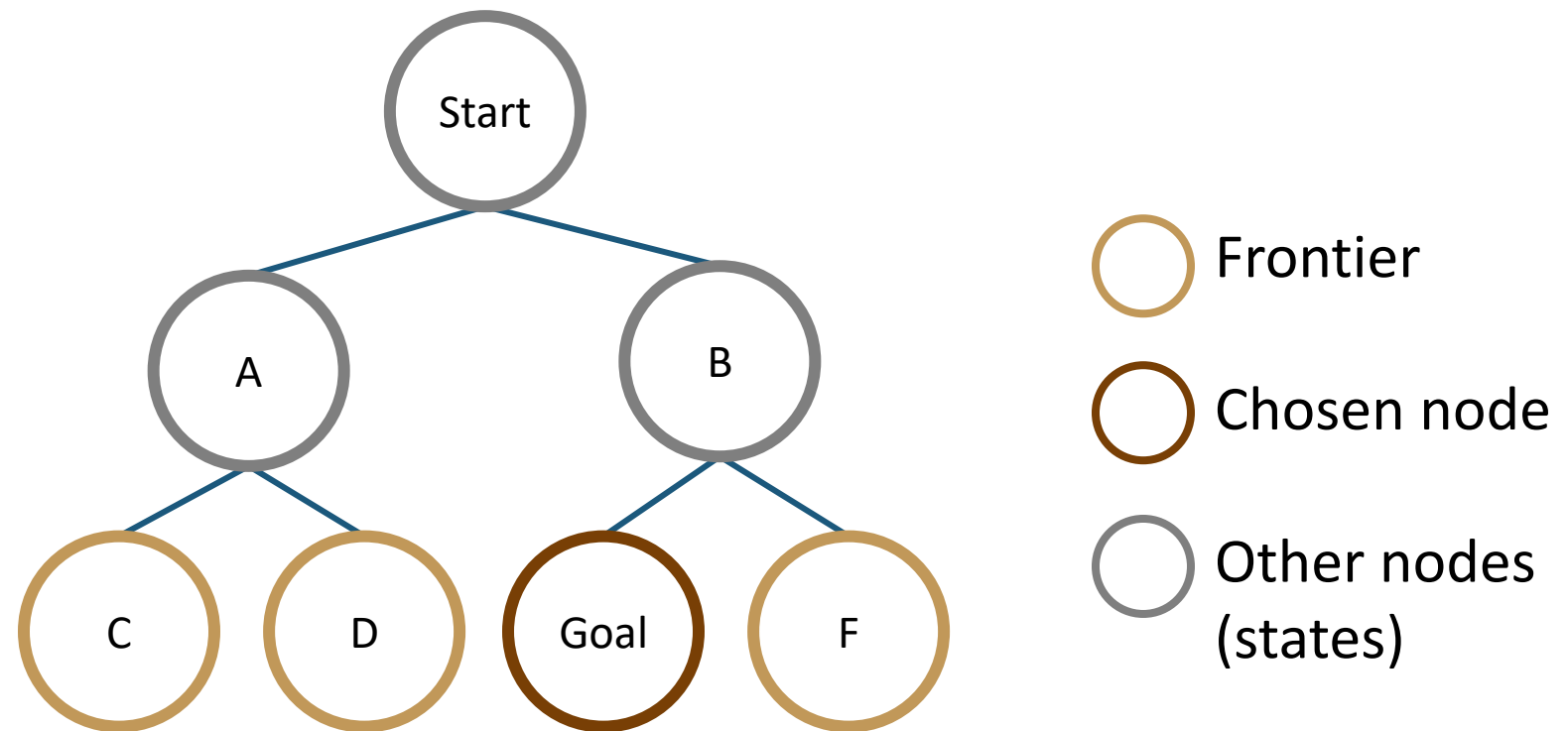
The Generic Search Algorithm



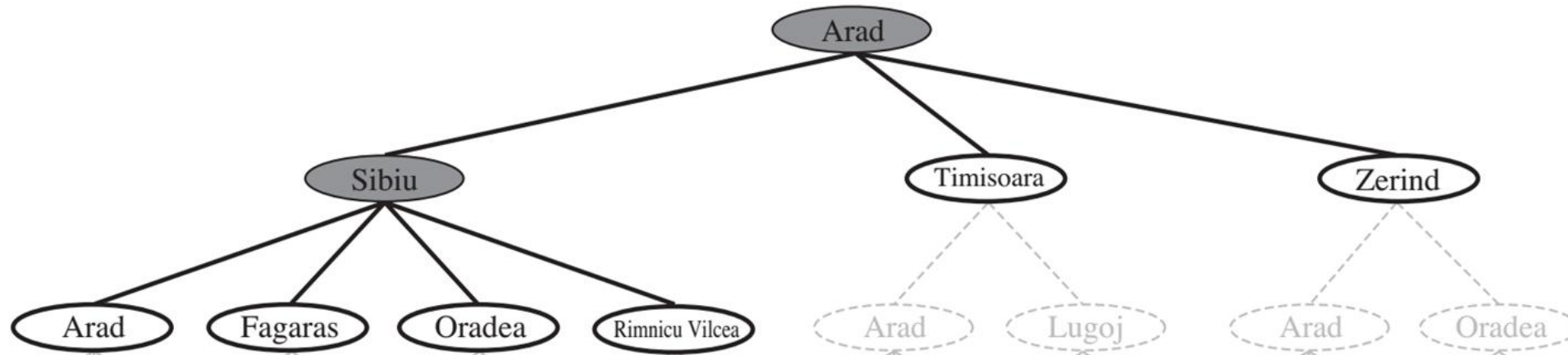
The Generic Search Algorithm



The Generic Search Algorithm



Generalized tree search



function TREE-SEARCH(*problem*, *strategy*) return a solution or failure

Initialize frontier to the *initial state* of the *problem*
do

if the frontier is empty then return *failure*

choose leaf node for expansion according to strategy & remove from frontier

if node contains goal state then return *solution*

else expand the node and add resulting nodes to the frontier

The strategy determines search process!

The Generic Search Algorithm

function TREE-SEARCH(*problem*) **returns** a solution, or failure

 initialize the frontier using the initial state of *problem*

loop do

if the frontier is empty **then return** failure

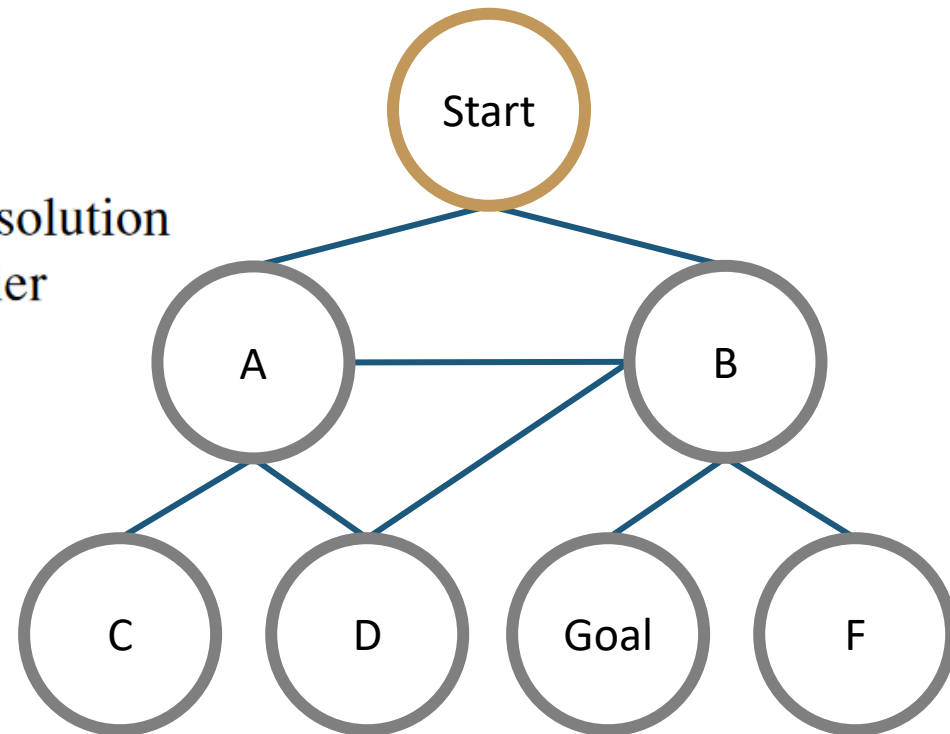
 choose a leaf node and remove it from the frontier

if the node contains a goal state **then return** the corresponding solution

 expand the chosen node, adding the resulting nodes to the frontier

Challenges:

- Loops
- Inefficiency

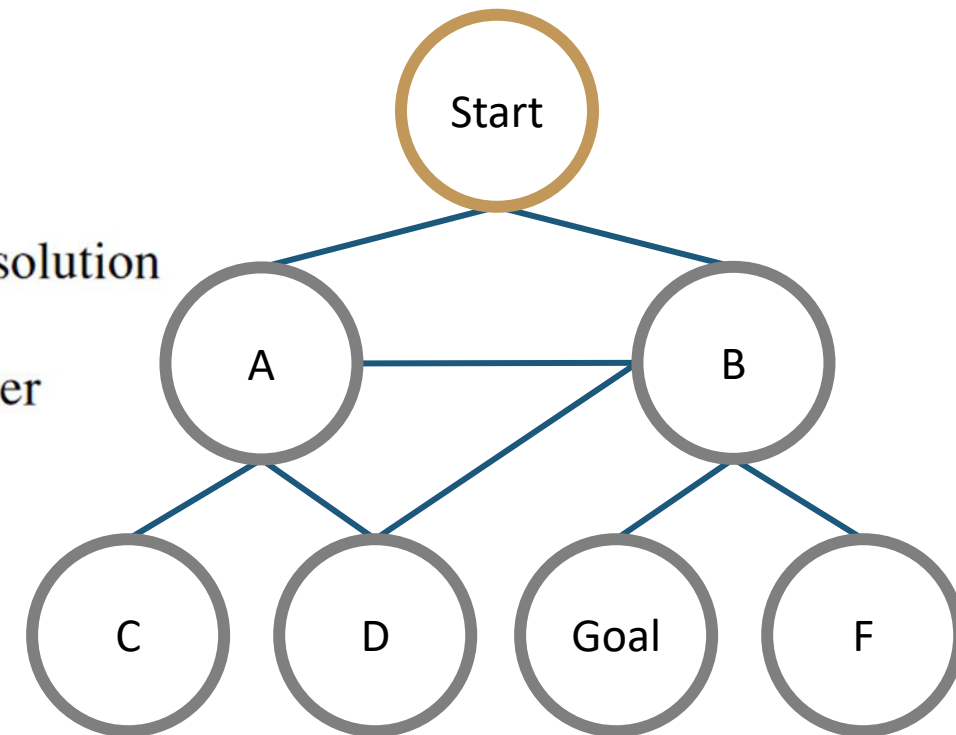


The Generic Search Algorithm

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
 initialize the frontier using the initial state of *problem*
 initialize the explored set to be empty
 loop do
 if the frontier is empty **then return** failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state **then return** the corresponding solution
 add the node to the explored set
 expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set

Challenges:

- ~~Loops~~
- Inefficiency



Search Strategies

Several classic search algorithms differ only by the order of how they expand their search trees

You can implement them by using different **frontier data structures**

- **Depth-first search** = LIFO queue
- **Breadth-first search** = FIFO queue
- **Greedy best-first search** or **A* search** = Priority queue

What about Efficiency?

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

Use data structures to control which nodes get explored next!

How we store the frontier affects the performance of the algorithm.

Breadth-First Search (BFS)

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

Stores frontier as first-in first-out (FIFO) queue

Properties of Breadth-First Search (BFS)

Complete?

Yes (if b is finite)

Optimal?

Yes, if cost = 1 per step
(not optimal in general)

Time Complexity?

$1+b+b^2+b^3+\dots+b^d = O(b^d)$

Space Complexity?

$O(b^d)$ (keeps every node in memory)

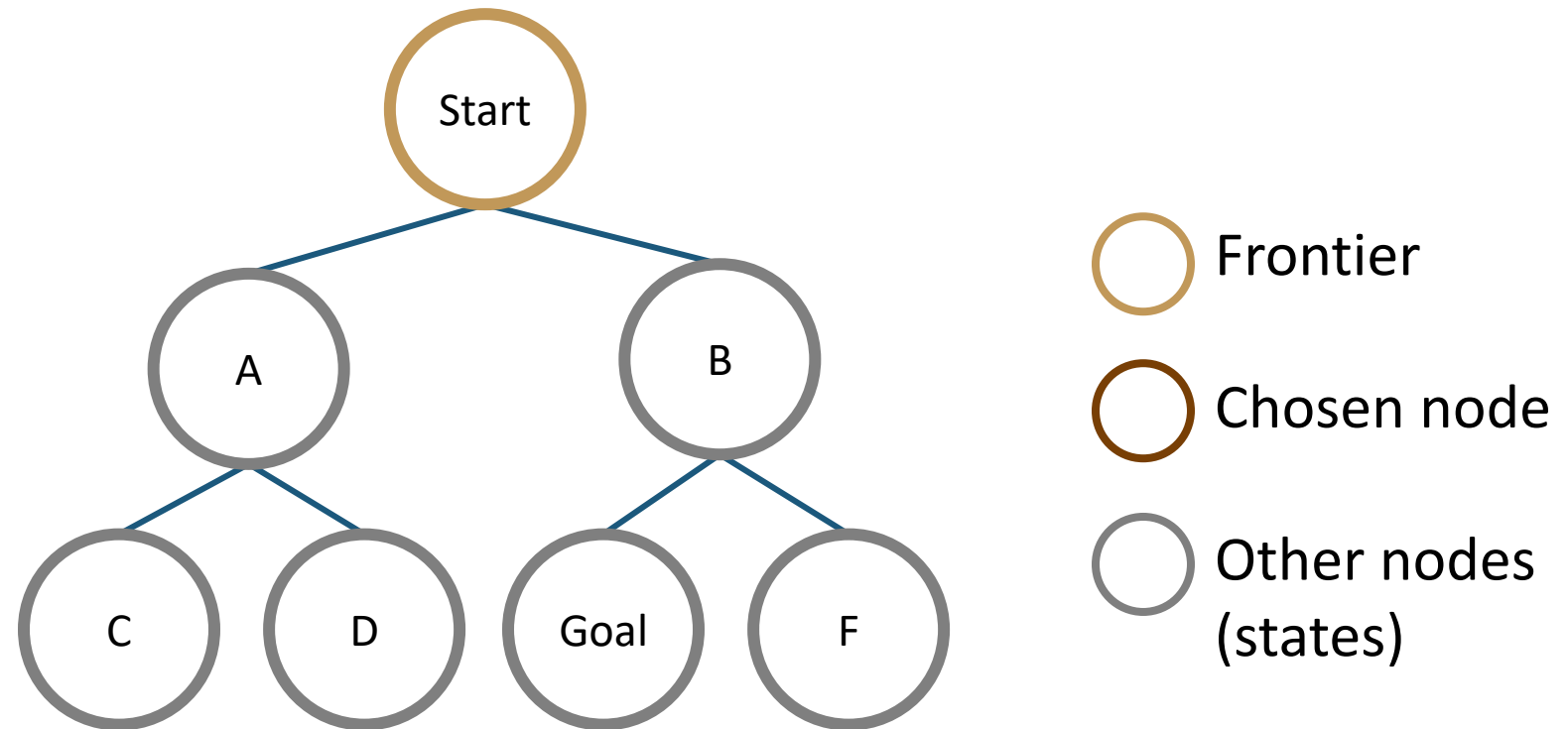
Time/space complexity variables

- b , **maximum branching factor** of search tree
- d , **depth** of the shallowest goal node
- m , **maximum length** of any path in the state space (potentially ∞)

Breadth-First Search (BFS): FIFO Queue

Frontier:
[Start]

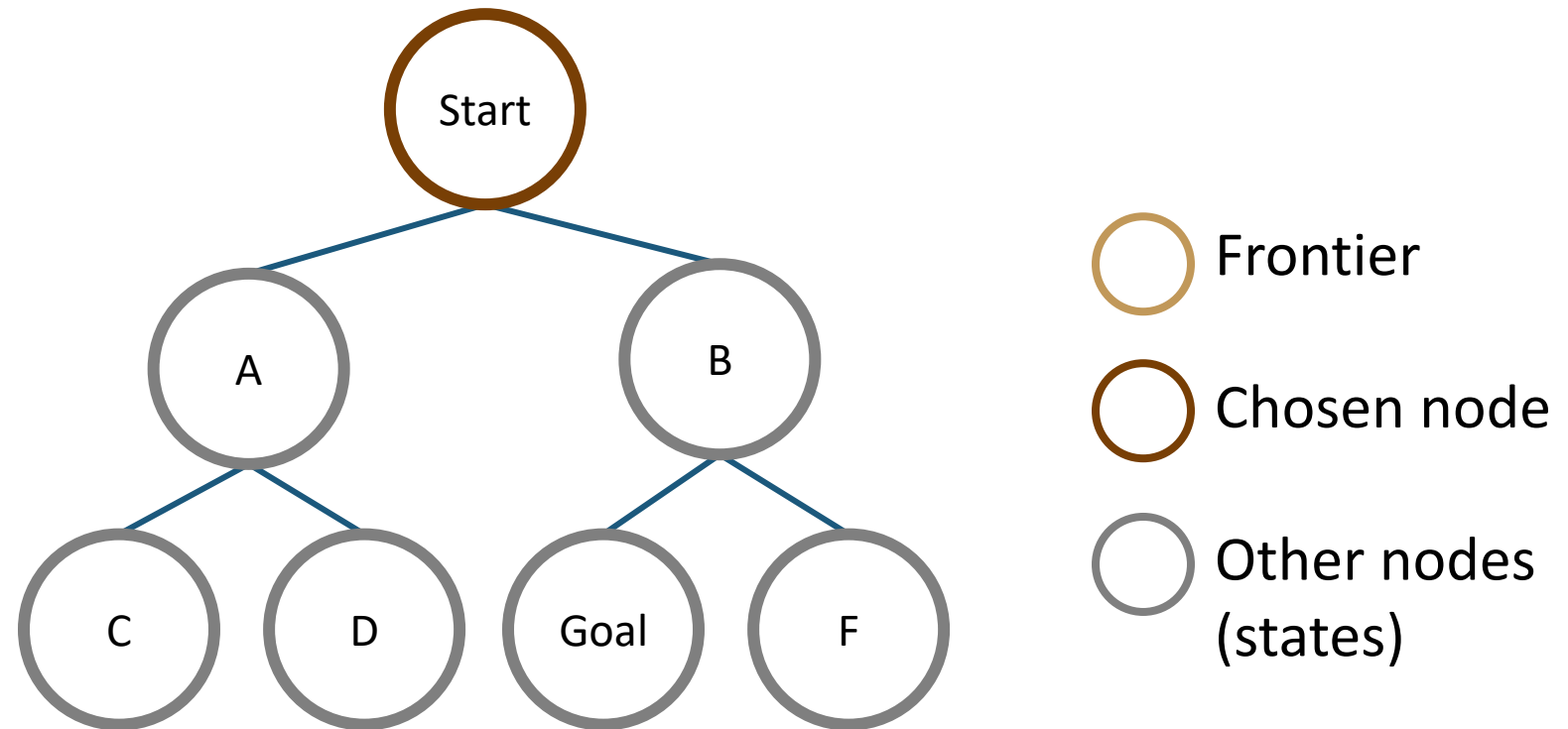
Current node:
none



Breadth-First Search (BFS): FIFO Queue

Frontier:
[]

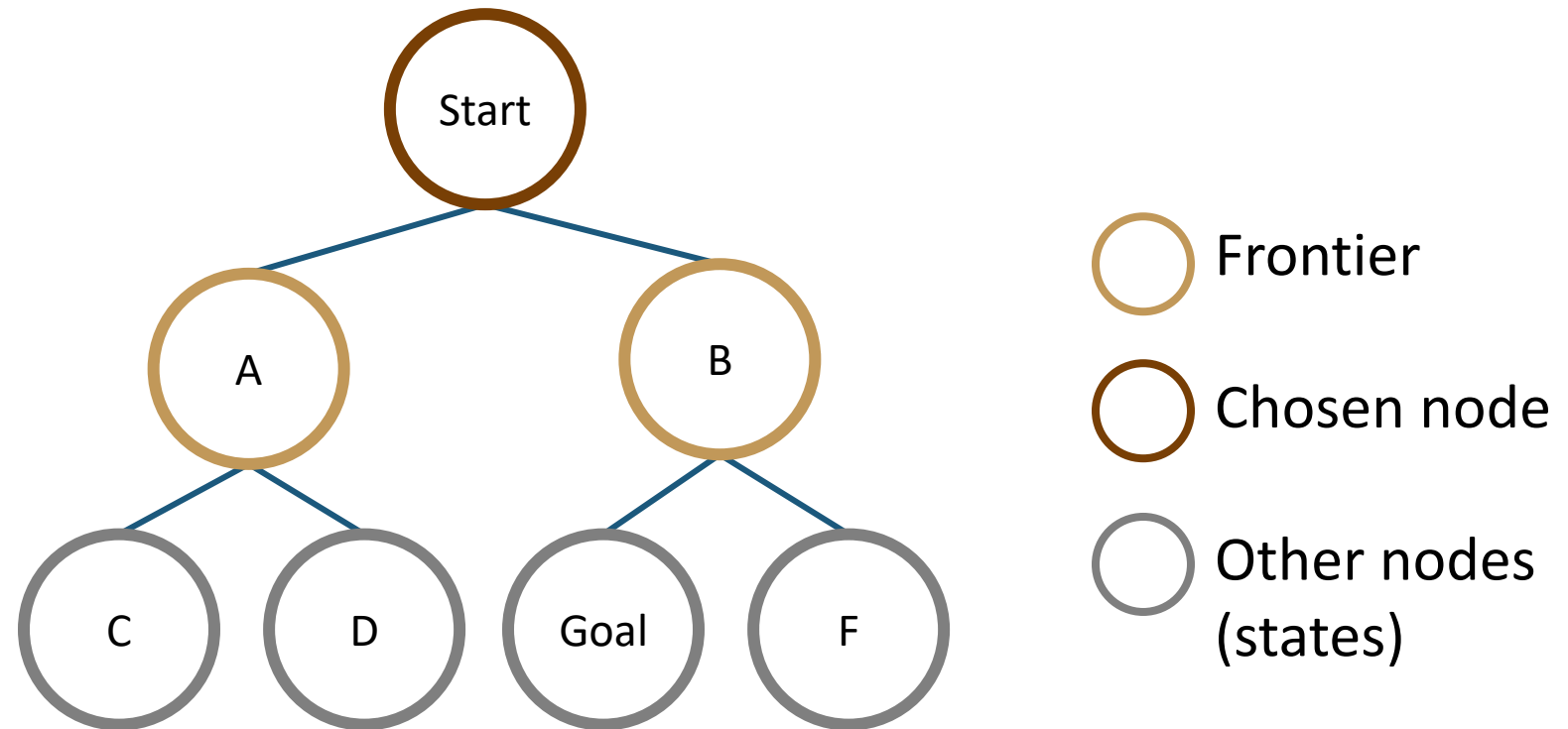
Current node:
Start



Breadth-First Search (BFS): FIFO Queue

Frontier:
[A, B]

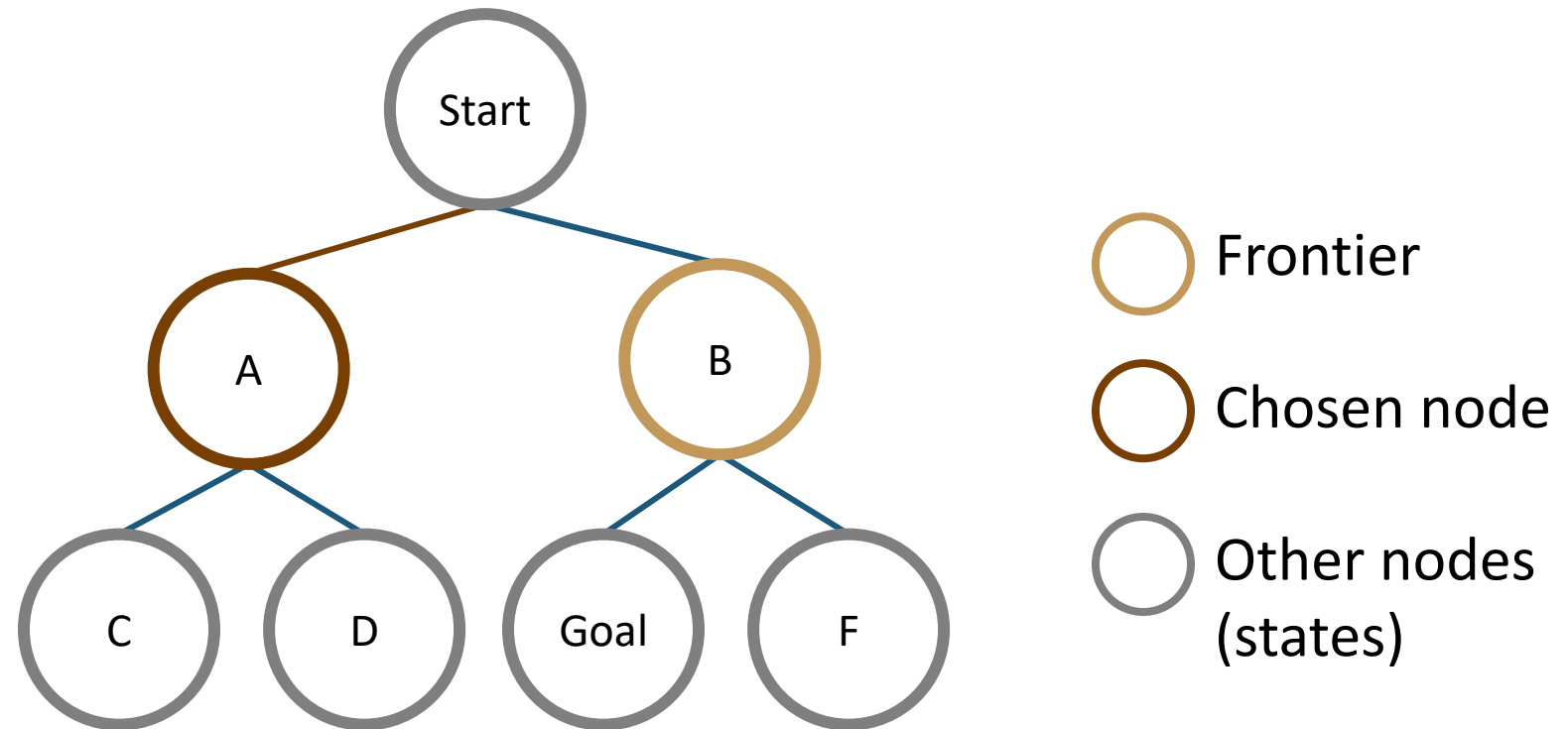
Current node:
Start



Breadth-First Search (BFS): FIFO Queue

Frontier:
[B]

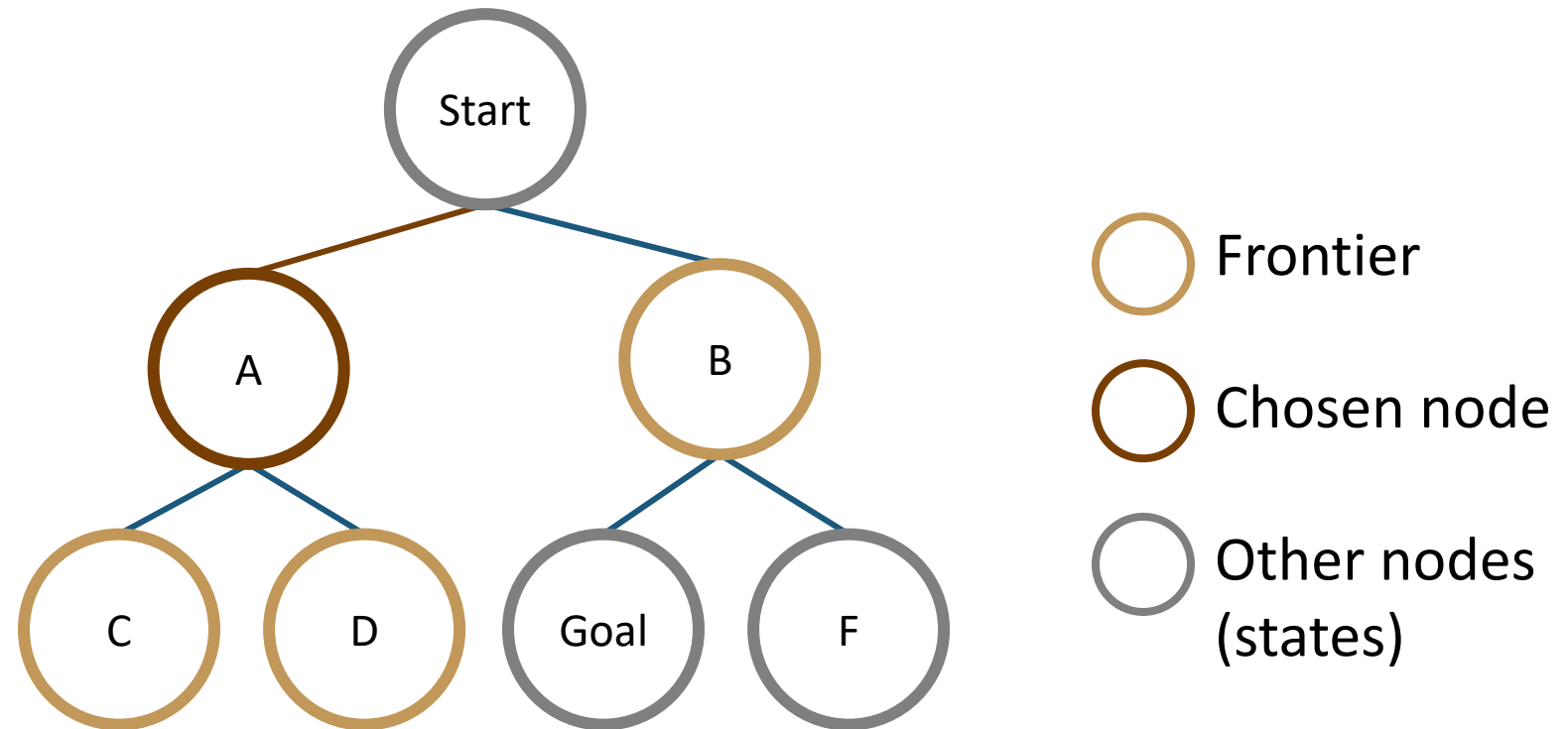
Current node:
A



Breadth-First Search (BFS): FIFO Queue

Frontier:
[B, C, D]

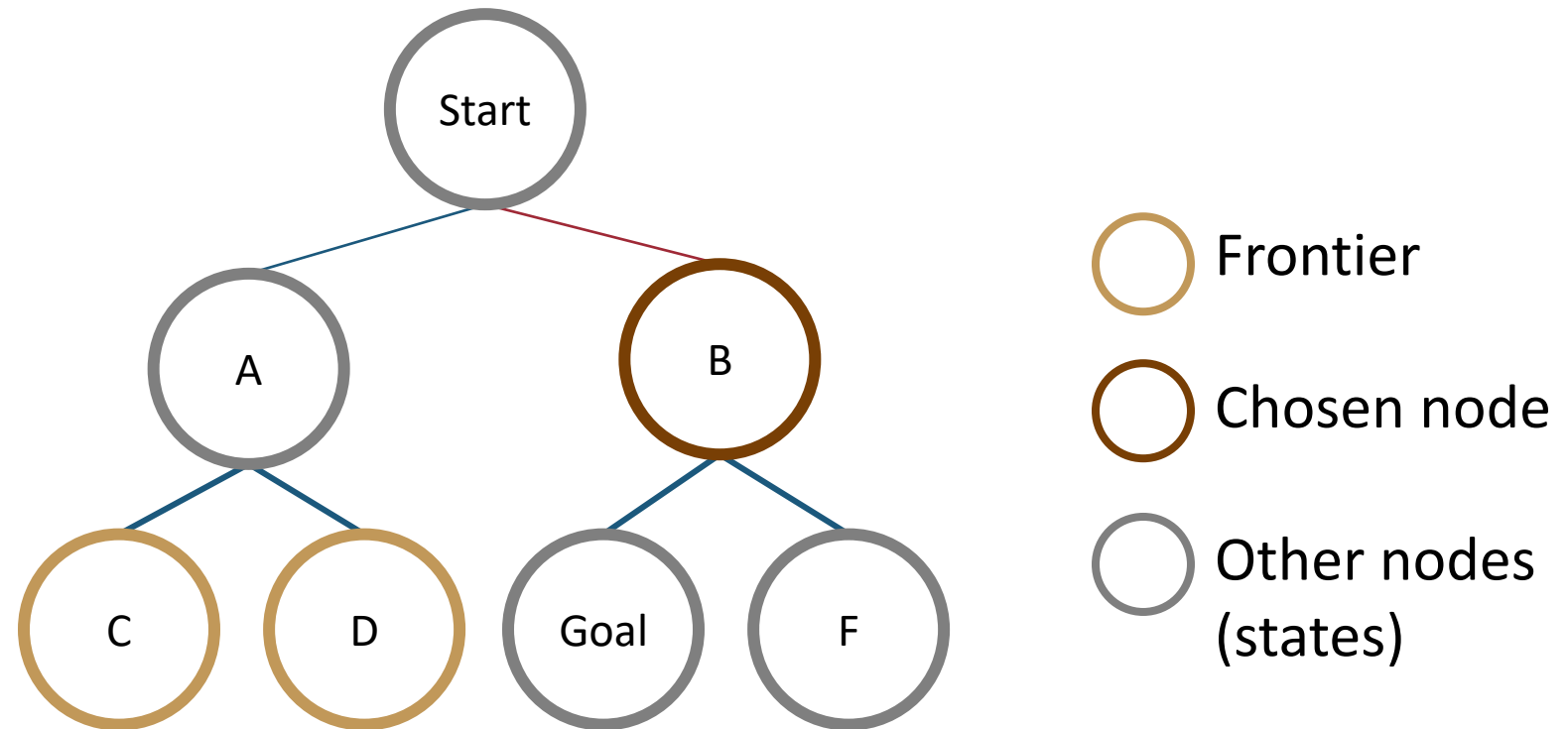
Current node:
A



Breadth-First Search (BFS): FIFO Queue

Frontier:
[C, D]

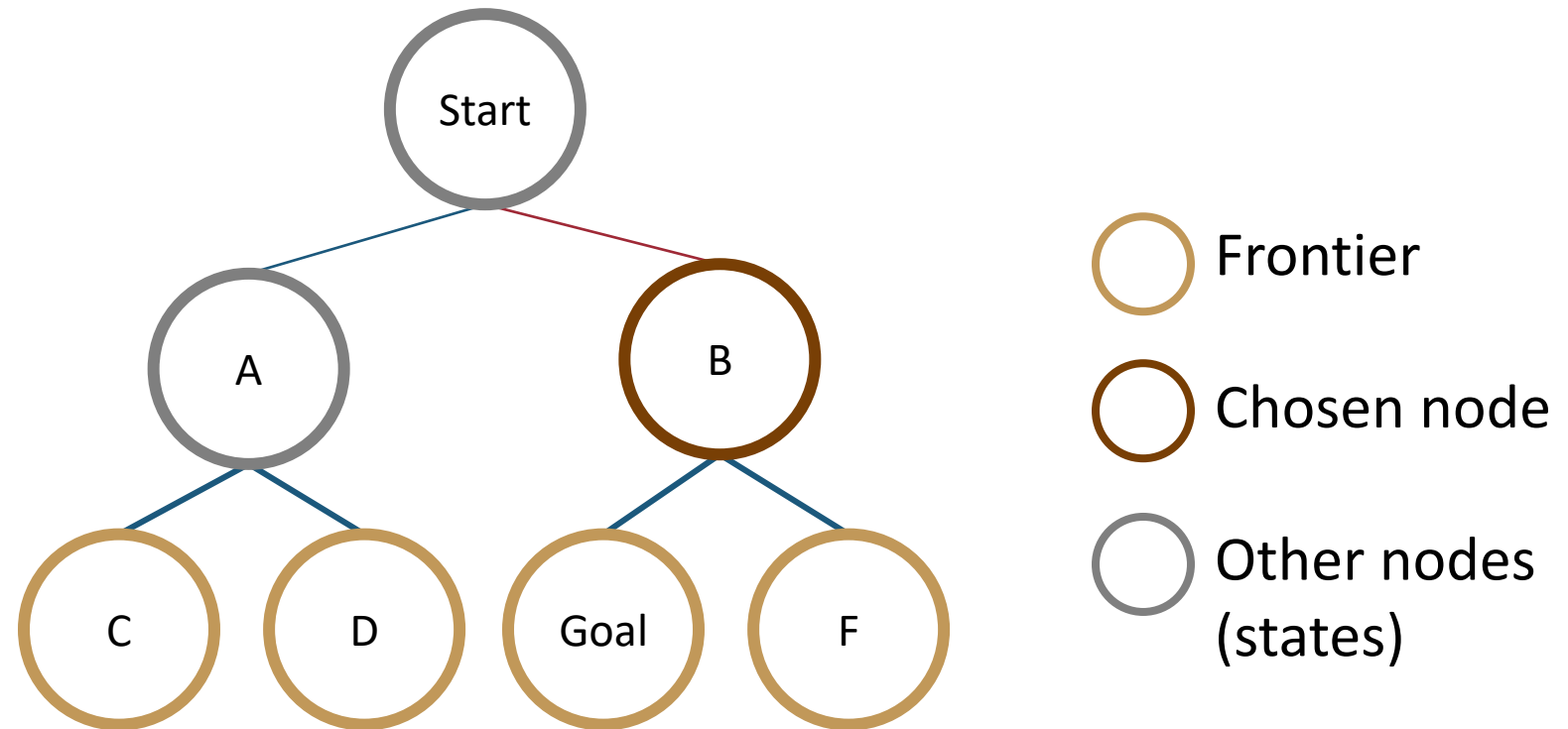
Current node:
B



Breadth-First Search (BFS): FIFO Queue

Frontier:
[C, D, Goal, F]

Current node:
B



Breadth-First Search (BFS): FIFO Queue

Frontier:

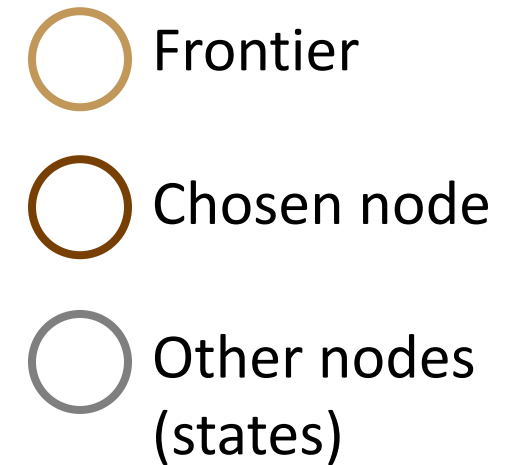
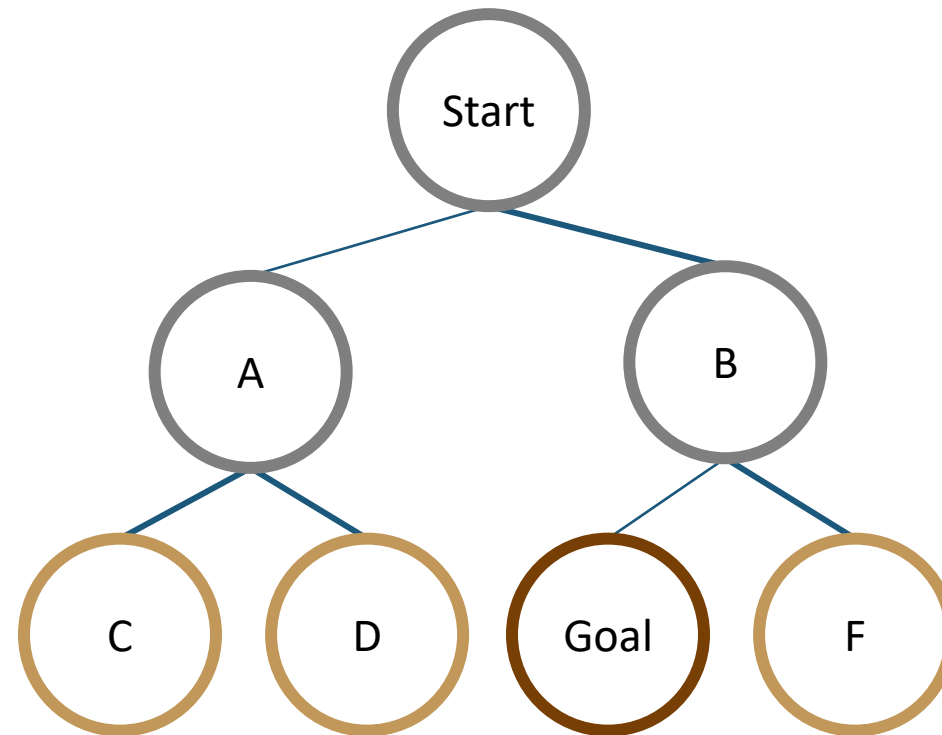
[C, D, Goal, F]

Eventually...

Current node:
B

Return:

[Start, B, Goal]



Depth-First Search (DFS)

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

Stores frontier as last-in first-out (LIFO) queue, or a stack.

BFS versus DFS

Breadth-first

- ✓ Complete,
- ✓ Optimal
- ✗ *but* uses $O(b^d)$ space

Depth-first

- ✗ Not complete unless m is bounded
- ✗ Not optimal
- ✗ Uses $O(b^m)$ time; terrible if $m \gg d$
- ✓ *but* only uses $O(b*m)$ space

Time/space complexity variables

b , *maximum branching factor* of search tree
 d , *depth* of the shallowest goal node
 m , *maximum length* of any path in the state space (potentially ∞)

Your Turn – Depth-First Search: LIFO Stack

Frontier:
[Start]

Current node:
none

Perform DFS on this search tree, listing the frontier and current node at every step until you reach the goal. Please submit to Blackboard when you're done.

